



**UNIVERSITY OF
GREENWICH**
London | Egypt

October University for Modern Sciences and Art

Faculty of Computer Science

Graduation Project

QuantWise: An AI-Powered Decision-Support Stock Advisory Platform

*Personalised, Risk-Graded Investment Recommendations for Retail
Investors*

Team Members	[Seif ElDein Mostafa — 235057, Yahia Ahmed — 235161]
Supervisor	[Dr. Marwa Solayman]
Co-Supervisor (TA)	[Eng. Farah Darwish]
Academic Year	2025 / 2026
Submission Date	June 18, 2026

Requirements for the B.Sc. in Computer Science — SE major

Abstract

Retail investors today face a frustrating paradox: financial markets produce more data than anyone can meaningfully process, yet personalised, actionable investment guidance remains largely out of reach for those without a professional background. This dissertation presents **QuantWise**, a decision-support stock advisory platform built for non-expert investors who want to make informed, risk-aware decisions without needing a finance degree.

The platform is structured as a polyglot, multi-tier system. A machine-learning pipeline scores the top United States equities each night, producing price-direction predictions enriched with news sentiment and analyst consensus data. A deterministic risk-grading engine then classifies each stock by risk level and conviction score. At request time, a modular-monolith backend calls a large language model to personalise those market-wide signals against the authenticated user's risk profile, returning a ranked set of BUY, SELL, and HOLD recommendations with per-pick allocation guidance and plain-language rationale. A React-based frontend brings all of this together alongside portfolio management.

Core functionality was validated through 68 automated unit and integration tests covering the primary domain workflows. The result shows that pairing machine learning, natural language processing, and generative AI personalisation can produce interpretable, risk-graded investment guidance that is genuinely accessible to investors with little or no prior experience.

Keywords

Stock Portfolio Advisory, Machine Learning Pipeline, Large Language Model Personalisation, Modular Monolith Architecture, Risk-Graded Recommendations, Sentiment Analysis

Contents

Abstract	2
Keywords	2
1 Introduction	1
1.1 Background	1
1.1.1 The Retail Investment Landscape	1
1.1.2 Relevant Technical Domains	1
1.1.3 Software Engineering Context	2
1.2 Problem Statement	2
1.3 Motivation	3
1.4 Objectives	4
1.5 Scope	4
1.5.1 In Scope	4
1.5.2 Out of Scope	5
1.6 Expected Contribution	5
1.7 SDG Relevance	6
1.8 Thesis Organization	6
2 Literature Review / Background	8
2.1 Related Work	8
2.1.1 Theme 1: Deep Learning for Financial Time-Series Prediction	8
2.1.2 Theme 2: Sentiment Analysis in Financial Markets	9
2.1.3 Theme 3: Large Language Models Applied to Finance	9
2.1.4 Theme 4: Commercial Robo-Advisory Platforms	10
2.2 Comparison of Existing Approaches	10
2.3 Research Gap	12
2.4 Theoretical Background	12
2.4.1 Long Short-Term Memory Networks	13
2.4.2 Gradient-Boosted Trees (XGBoost)	13
2.4.3 Transformer Architecture and FinBERT	13
2.4.4 Command/Query Responsibility Segregation (CQRS)	14
2.4.5 Transactional Outbox and Inbox Patterns	14
2.4.6 Modular Monolith Architecture	14

2.5	Datasets & Technologies	15
2.5.1	Market Data: yFinance and Yahoo Finance	15
2.5.2	Analyst and News Data: Finnhub API	15
2.5.3	Backend Framework: ASP.NET Core and MediatR	15
2.5.4	Machine Learning Runtime: PyTorch and XGBoost	15
2.5.5	Generative AI: Google Gemini API	16
2.5.6	Infrastructure: PostgreSQL, Redis, and RabbitMQ	16
3	Software Requirements & Project Management	17
3.1	Stakeholders & Users	17
3.1.1	Organisational Stakeholders	17
3.1.2	End-User Personas	17
3.1.3	System Actors	18
3.2	Functional Requirements	19
3.3	Non-Functional Requirements (NFR)	22
3.4	Use Cases & User Stories	24
3.4.1	Detailed Use Cases	26
3.4.2	Representative User Stories	29
3.5	System Constraints & Assumptions	30
3.6	Requirements Traceability	30
3.7	Project Management Methodology & Task Allocation	32
3.7.1	Why Kanban Rather Than Scrum	32
3.7.2	Project Scope	33
3.7.3	Team Roles and Work Breakdown	33
3.8	Schedule & Progress Tracking	34
3.8.1	Progress-Tracking Method	35
3.9	Risk Management & Mitigation Strategy	36
3.10	Communication Plan	37
4	System Design and Implementation	38
4.1	System Architecture Overview	38
4.2	Backend Architecture: the Modular Monolith	39
4.2.1	Layers and the Dependency Rule	39
4.2.2	CQRS with MediatR	40
4.2.3	Module Boundaries	40
4.2.4	Asynchronous Messaging: Outbox, Inbox, and Quartz	41
4.3	The Prediction Pipeline	41
4.3.1	Deterministic Risk Grading	43
4.4	Detailed UML Design	43
4.4.1	Domain Class Model	43

4.4.2	Key Sequence Diagrams	44
4.4.3	Onboarding Activity	46
4.5	Database / Data Storage Design	46
4.6	UI/UX Design	47
4.7	Technology Stack Justification	50
4.8	Core Modules & Code Snippets	51
4.8.1	Grounding the Generative Model	51
4.8.2	The Recommendation Query Handler	53
4.8.3	Idempotent Ingestion	54
4.8.4	The Outbox Interceptor	54
4.8.5	The Pipeline Core	55
5	Software Testing and Evaluation	57
5.1	Test Strategy & Testing Environment	57
5.2	Unit & Integration Testing	58
5.2.1	Unit Tests	58
5.2.2	Integration Tests	59
5.2.3	Coverage	59
5.3	System & Black-Box Test Cases	60
5.4	Bug Tracking, Defect Lifecycle & Re-testing	62
5.5	Usability & User Acceptance Testing	63
5.6	Non-Functional Evaluation	65
5.6.1	Performance	65
5.6.2	Behaviour under load	66
5.6.3	Security	67
5.6.4	Reliability	67
5.7	Requirements Coverage Summary	68
6	Conclusion and Future Work	70
6.1	Summary of the Developed Product	70
6.2	Achievement of Objectives and Requirements	71
6.3	Testing and Evaluation Conclusions	71
6.4	Engineering Limitations	72
6.5	Future Enhancements	73
6.6	Concluding Remarks	74
	Individual Contributions	75
	References	78
	Appendices	79
	Appendix B — Interface Screenshots	79

List of Figures

3.1	QuantWise use-case diagram.	25
3.2	Project schedule as a Gantt chart.	35
4.1	High-level system architecture of QuantWise.	38
4.2	Deployment topology under Docker Compose.	39
4.3	Backend module and layer structure.	40
4.4	Daily scoring flow inside the prediction pipeline.	42
4.5	Domain class diagram across the four modules.	44
4.6	Sequence: personalised recommendation request (cache-miss path). . .	45
4.7	Sequence: daily ingest and notification fan-out.	45
4.8	Sequence: login and JWT issuance.	45
4.9	Activity: onboarding risk-profile calculation.	46
4.10	Entity-relationship diagram across the four module schemas.	47
4.11	QuantWise landing page (the “Quant Terminal” design system).	49
4.12	Dashboard: personalised BUY/SELL/HOLD recommendations with dollar allocations.	50
5.1	Read-path latency under concurrent load (50 virtual users).	67
6.1	Authentication and onboarding.	79
6.2	Dashboard, market, learning, and profile screens.	80

List of Tables

2.1	Comparison of existing approaches against QuantWise requirements.	11
3.1	Stakeholder and user interactions with the platform.	18
3.2	Functional requirements with MoSCoW prioritisation.	19
3.3	Non-functional requirements with measurable targets.	23
3.4	UC-02 — Authenticate (Log In).	26
3.5	UC-03 — Complete the Risk Questionnaire.	26
3.6	UC-04 — View Personalised Recommendations.	27
3.7	UC-07 — Ingest the Daily Run and Notify Users (system).	28
3.8	Summary of the remaining use cases.	29
3.9	Requirements traceability matrix.	31
3.10	Work breakdown structure by epic.	33
3.11	Project milestones reconstructed from the GitHub history.	34
3.12	Risk register and mitigation.	36
5.1	Testing environment and tooling.	58
5.2	Automated test counts by project.	58
5.3	Code coverage (coverlet + ReportGenerator).	60
5.4	System and black-box test cases.	60
5.5	Defect log with resolution and re-test.	62
5.6	SUS scores by participant ($n = 10$).	64
5.7	Task-completion results ($n = 10$).	65
5.8	Measured response times (20 samples per endpoint).	66
5.9	Read-path latency at 50 concurrent users (~ 103 req/s, 12,464 requests, 0% errors).	66
5.10	Requirements-to-evidence coverage.	68
6.1	Objective achievement against delivered evidence	71
6.2	Non-functional requirement status	72

Chapter 1

Introduction

Retail participation in global financial markets has grown dramatically over the past decade, and so has the volume of publicly available data that investors are expected to make sense of. Individual investors today are simultaneously better-informed and more overwhelmed than at any previous point in financial history. This chapter sets out the domain and motivation behind **QuantWise**, a decision-support stock advisory platform that combines machine learning, natural language processing, and generative AI to deliver personalised, risk-graded investment recommendations to non-expert retail investors.

1.1 Background

1.1.1 The Retail Investment Landscape

Retail investing has been through a structural shift over the past decade. Commission-free platforms such as Robinhood and eToro have pushed the barrier to market entry to near zero, and tens of millions of first-time investors have taken them up on it. According to the Financial Industry Regulatory Authority (FINRA), the number of individual brokerage accounts in the United States grew by more than 30% between 2019 and 2022, with comparable trends in European and Middle Eastern markets.

Access to quality *decision support*, however, has not kept pace. Professional portfolio management draws on quantitative financial modelling, macroeconomic analysis, real-time news sentiment interpretation, and structured risk profiling — disciplines that demand significant expertise and are traditionally delivered only through expensive wealth-management services. Without that support, retail investors are left navigating highly volatile markets with either generic robo-advisors that apply static allocation rules or purely manual research that simply cannot scale to the breadth of modern equity markets — a gap that leads, predictably, to avoidable losses.

1.1.2 Relevant Technical Domains

Three converging technical fields make QuantWise possible:

- **Financial Time-Series Forecasting.** Deep learning architectures, specifically

Long Short-Term Memory (LSTM) recurrent neural networks and gradient-boosted ensemble models, have demonstrated strong performance in predicting short-horizon equity price movements by learning non-linear temporal dependencies in historical Open-High-Low-Close-Volume (OHLCV) data alongside technical indicators.

- **Financial Sentiment Analysis.** Transformer-based language models pre-trained on financial corpora, such as FinBERT, have shown strong performance in classifying the sentiment polarity of financial news headlines and analyst reports. Incorporating sentiment signals alongside quantitative predictions has been shown to improve directional accuracy and reduce false-positive trade signals.
- **Large Language Model Personalisation.** Generative AI models such as Google Gemini provide the capability to synthesise structured numerical signals and unstructured rationale into coherent, personalised natural-language recommendations, enabling a system to adapt a market-wide signal set to the individual risk profile of each user at request time.

1.1.3 Software Engineering Context

From an engineering perspective, QuantWise is a multi-tier, polyglot system with a dual operational personality: it has to satisfy the throughput demands of a nightly batch machine-learning pipeline and the latency expectations of a consumer-facing web application at the same time. The backend is implemented as a *modular monolith* on ASP.NET Core — a pattern that delivers the maintainability benefits of strong vertical separation without the operational overhead that comes with a full microservices deployment. The frontend is a React single-page application, and the prediction pipeline is a Python FastAPI service that encapsulates all ML inference and data acquisition logic. Cross-module communication is handled asynchronously through RabbitMQ, using the Outbox/Inbox transactional messaging pattern to guarantee at-least-once delivery without creating tight runtime coupling between services.

1.2 Problem Statement

Core problem: Non-expert retail investors have no access to personalised, risk-aware, data-driven investment guidance that is both computationally grounded and interpretable — so they make critical financial decisions without adequate support.

The problem has three distinct faces:

- **Information overload without interpretation.** Equity markets generate enormous volumes of price data, analyst reports, and news events every single day. A retail investor tracking even a small set of stocks cannot manually process that volume, let alone translate it into structured, actionable signals.
- **Generic, non-personalised advisory outputs.** Consumer-facing robo-advisors

apply fixed asset-allocation rules without incorporating real-time market signals, individual risk nuance, or per-stock directional recommendations. They advise on *asset classes*, not *specific equities*.

- **Absence of explainability.** Even accurate algorithmic signals are not actionable for a non-expert without accompanying rationale. An investor who does not understand *why* a recommendation was made is unlikely to act on it with the right level of confidence — or the right level of caution.

The stakeholders most directly affected are retail investors who hold self-directed brokerage accounts but lack the expertise or tooling to perform systematic, risk-adjusted stock selection. Indirectly, financial advisory firms looking to automate parts of their guidance workflow, and regulators who benefit from investors making better-informed decisions, are also in the picture.

No existing alternative resolves all three dimensions at once. Algorithmic trading platforms target professionals and provide no plain-language rationale; generic robo-advisors lack equity-level signals; and manual research tools such as Bloomberg Terminal or Reuters Eikon are prohibitively expensive and require expert interpretation to use effectively.

1.3 Motivation

Three converging trends made building QuantWise feel both timely and necessary:

- **Rising retail market participation.** Retail investors accounted for roughly 23% of US equity trading volume in 2023, up from under 15% in 2019. As this segment keeps growing, the absence of proper decision-support tooling represents a market failure with real financial consequences for a large and expanding population.
- **Maturity of applicable AI techniques.** LSTM-based time-series models, transformer sentiment classifiers, and large language models have all reached a level of practical reliability, open-source availability, and inference efficiency that makes integrating them into a production-grade web platform genuinely feasible — without needing proprietary data infrastructure or prohibitive compute costs.
- **The alignment challenge in generative AI advisory.** General-purpose large language models have generated real interest as a vehicle for investment advice. The problem is that ungrounded model responses — ones that ignore a user’s verified risk profile or current market signals — carry significant reliability risks. QuantWise addresses this directly by anchoring LLM personalisation to a structured, validated quantitative signal set, keeping the model in the role it does best: synthesising and explaining pre-computed data rather than performing raw financial reasoning from scratch.

1.4 Objectives

Primary objective: Design, build, and evaluate a full-stack decision-support platform that delivers daily, personalised, risk-graded stock recommendations to retail investors by integrating a machine-learning prediction pipeline, financial sentiment analysis, and large language model personalisation.

Specific technical objectives:

- O1. Design** a nightly machine-learning pipeline that ingests historical OHLCV price data and technical indicators for the top 100 US large-capitalisation equities, deploys a two-stage LSTM and XGBoost ensemble, and produces directional price predictions with associated confidence scores.
- O2. Formulate** a composite sentiment analysis scoring system that aggregates FinBERT-classified news headlines, Finnhub analyst consensus ratings, and analyst price targets into a single weighted sentiment signal per equity.
- O3. Implement** a deterministic risk-grading engine that cross-validates quantitative predictions against sentiment signals to classify each stock as Low, Medium, or High risk and assign a numerical conviction score, ensuring reproducible and auditable risk decisions.
- O4. Build** a modular-monolith backend in ASP.NET Core implementing Command/-Query Responsibility Segregation (CQRS) via MediatR, with independent database contexts per module and asynchronous event propagation via RabbitMQ and the Outbox/Inbox pattern.
- O5. Integrate** Google Gemini at request time to personalise market-wide risk-graded signals against each authenticated user's risk profile (Conservative, Moderate, or Aggressive), producing ranked BUY, SELL, and HOLD picks with per-pick allocation guidance and plain-language rationale, cached in Redis for twenty-four hours.
- O6. Validate** the system through unit and integration tests covering the core domain command handlers, risk-profile resolution logic, and LLM response parsing, and conduct a structured usability evaluation of the React-based frontend with target-user participants.

1.5 Scope

1.5.1 In Scope

- **Equity universe:** The top 100 US large-capitalisation stocks listed on NASDAQ and NYSE, dynamically resolved via the yFinance EquityQuery screener with a

hardcoded fallback list.

- **Prediction horizon:** Next-day (1-day ahead) directional price movement (UP / DOWN) with an associated percentage change estimate and confidence score.
- **User modules:** Registration and authentication (JWT), onboarding risk questionnaire, portfolio creation and management, personalised AI recommendations dashboard, and in-app notification system.
- **Recommendation delivery:** BUY, SELL, and HOLD signals with allocation percentage guidance and natural-language rationale, personalised per user risk profile.
- **Platform:** Web application accessible via modern desktop browsers. Backend API hosted locally or on a single-server deployment via Docker Compose.
- **Testing:** Unit and integration tests for backend domain logic; usability evaluation for the frontend.

1.5.2 Out of Scope

- **Automated trade execution:** The platform provides decision support only. No brokerage API integration or automated order placement is implemented.
- **Non-US equities:** International markets, ETFs, bonds, commodities, and cryptocurrency are explicitly excluded from the prediction universe.
- **Intraday or high-frequency signals:** The pipeline runs once per day; real-time or tick-level predictions are not in scope.
- **Mobile applications:** Native iOS or Android clients are not developed; the frontend is a web-only interface.
- **Regulatory compliance certification:** The platform does not seek nor claim compliance with financial advisory regulations. It is an academic decision-support prototype.
- **Cloud-native deployment:** Horizontal auto-scaling, Kubernetes orchestration, and multi-region deployment are not implemented in this phase.

1.6 Expected Contribution

The central software engineering contribution of this project is a **grounded generative AI advisory architecture** — a system where the language model acts not as a raw reasoning engine, but as a structured personalisation and explanation layer operating over pre-computed, validated quantitative signals.

This design targets a reliability gap that has been consistently observed in prior generative AI applications to finance. By constraining the model to synthesise only signals already validated by the ML pipeline and risk-grading engine, the system sidesteps the hallucination and factual-drift risks that appear when language models are asked to do direct financial forecasting.

The project also delivers several additional engineering contributions:

- A **unified Python prediction service** combining LSTM-based time-series forecasting, XGBoost regression, and FinBERT sentiment classification inside a single FastAPI endpoint with deterministic risk grading, eliminating the need for separate orchestration infrastructure.
- A **modular-monolith backend architecture** in .NET that shows how vertical module separation, per-module database contexts, and transactional outbox messaging can achieve strong module-level isolation without the operational overhead of a service mesh.
- A **Redis caching strategy** for LLM-generated recommendation payloads, offering a practical and cost-effective way to integrate generative AI into latency-sensitive consumer request paths.

1.7 SDG Relevance

QuantWise aligns with three United Nations Sustainable Development Goals:

- **SDG 8 — Decent Work and Economic Growth.** Providing retail investors with data-driven, risk-graded guidance that was previously accessible only through paid professional services supports inclusive economic participation and productive engagement with financial markets.
- **SDG 10 — Reduced Inequalities.** Quality investment advisory has historically been stratified by wealth. QuantWise democratizes access to personalised, algorithmically grounded stock guidance, narrowing the informational gap between retail and institutional investors regardless of a user’s financial literacy or economic background.
- **SDG 9 — Industry, Innovation, and Infrastructure.** The platform integrates state-of-the-art AI techniques — deep learning, transformer-based NLP, and generative AI — within a production-grade software engineering architecture, contributing to innovative, resilient digital financial infrastructure.

1.8 Thesis Organization

The rest of this dissertation is structured as follows:

- **Chapter 2 — Literature Review / Background** surveys related work in financial time-series forecasting, sentiment-augmented prediction models, and AI-driven advisory systems. It identifies the gap in existing work that motivates the QuantWise architecture, and lays out the theoretical and technological foundations needed to understand the implementation.
- **Chapter 3 — Software Requirements & Project Management** presents the

complete Software Requirements Specification — functional and non-functional requirements with MoSCoW prioritisation, use-case descriptions, system constraints, project management approach, timeline, and risk register.

- **Chapter 4 — System Design and Implementation** documents the full engineering design of the platform: multi-tier system architecture, detailed UML diagrams (class, sequence, and activity), the entity-relationship database schema, UI/UX design, technology stack justification, and annotated core code modules.
- **Chapter 5 — Software Testing and Evaluation** presents a multi-tier test execution report covering unit, integration, system, and usability testing. Every test case is traced back to a requirement from Chapter 3, and non-functional evaluation results are presented with performance and coverage metrics.
- **Chapter 6 — Conclusion and Future Work** reflects on the delivered product, maps each objective to evidence, gives an honest account of engineering limitations, and lays out a concrete roadmap for future work.

Chapter 2

Literature Review / Background

This chapter reviews the research and commercial practice that most directly shapes the three technical pillars of QuantWise: financial time-series forecasting with deep learning, sentiment analysis applied to financial markets, and the use of large language models in financial advisory contexts. It closes by identifying the gap in that existing body of work that motivates our architecture, and by laying out the theoretical and technological foundations the implementation rests on.

2.1 Related Work

The literature relevant to QuantWise spans three thematically distinct but increasingly converging research areas. The discussion groups works by theme so the methodological strengths and limitations of each line of work can be compared directly.

2.1.1 Theme 1: Deep Learning for Financial Time-Series Prediction

The core forecasting problem that QuantWise’s prediction pipeline tackles — predicting equity price direction from historical OHLCV data — has been studied extensively using recurrent neural network architectures.

Fischer and Krauss [1] conducted a landmark study applying Long Short-Term Memory networks to predict out-of-sample directional price movements for all constituents of the S&P 500 index using data spanning 1992–2015. Their LSTM model achieved a mean daily return of 0.46% with a Sharpe ratio of 5.8 before transaction costs, significantly outperforming benchmark models including random forests, deep neural networks, and logistic regression. The study demonstrated that LSTM networks can capture temporal dependencies in financial time-series that simpler, memory-free models cannot. However, the work focused exclusively on price data without integrating sentiment or analyst signals, and produced no mechanism for personalising outputs to individual investor risk profiles. Furthermore, the authors noted that excess returns declined substantially after 2010, suggesting that price-only LSTM signals become less reliable as markets become more competitive.

Subsequent work extended the LSTM approach by incorporating technical indicators (RSI, MACD, moving averages) as additional input features alongside raw price

series, which has been shown to improve the signal-to-noise ratio of predictions in more recent market regimes [2]. QuantWise adopts this practice, using a 19-feature set that includes both normalised OHLCV inputs and 14 technical indicator features derived during the feature engineering stage.

2.1.2 Theme 2: Sentiment Analysis in Financial Markets

Textual information — news headlines, analyst reports, and social media — as a predictor of market movements has generated a rich body of research.

Bollen, Mao, and Zeng [3] showed that public mood expressed on Twitter, measured using a psychometric tool, could predict daily changes in the Dow Jones Industrial Average with roughly 87.6% accuracy when combined with a self-organising fuzzy neural network. The work established the predictive value of collective sentiment, though it relied on general social media mood rather than domain-specific financial discourse, which limits its precision for individual equity analysis.

Domain-specific transformer models raised the bar considerably. Araci [4] developed FinBERT, a language model built on the BERT architecture [5] and further pre-trained on a large corpus of financial communications. FinBERT outperformed prior methods on the Financial PhraseBank sentiment dataset, showing strong ability to classify financial text as positive, negative, or neutral even when fine-tuned on comparatively small labelled datasets. QuantWise uses the ProsusAI implementation of FinBERT (`ProsusAI/finbert`) to classify news headlines from Finnhub and Yahoo Finance, producing a per-ticker sentiment score that feeds into the composite sentiment signal.

A key limitation of sentiment-only approaches is that they do not account for the direction or magnitude of the underlying quantitative price movement. A positive sentiment signal for a stock the price model expects to fall is a contradiction — and flagging exactly that kind of conflict is central to the QuantWise risk-grading layer.

2.1.3 Theme 3: Large Language Models Applied to Finance

The use of both general-purpose and finance-specific large language models for investment guidance is one of the most rapidly evolving frontiers in the field.

Wu et al. [6] introduced BloombergGPT, a 50-billion parameter language model trained on over 700 billion tokens drawn from Bloomberg’s proprietary financial archives alongside general-purpose corpora. BloombergGPT significantly outperformed existing open-source models on finance-specific benchmarks including sentiment analysis, named entity recognition, and financial question answering. Its practical reach is limited, however: it is a proprietary model available only to Bloomberg Terminal subscribers and provides no mechanism for personalising outputs to individual risk profiles or integrating quantitative prediction signals.

Lopez-Lira and Tang [7] showed that GPT-4, prompted to evaluate the sentiment of financial news headlines, produced scores with statistically significant predictive power over subsequent daily stock returns, with portfolio-day hit rates of around 90% for non-tradable initial market reactions. The authors described this as an “emerging capacity” of sufficiently large models, absent in smaller predecessors such as GPT-2 or BERT. Importantly, the model in this study operates in a purely *generative reasoning* mode: it predicts market outcomes from text without access to any structured quantitative signals. QuantWise takes the opposite approach. Gemini is never asked to reason about market direction from scratch; instead, it receives pre-computed, validated quantitative and sentiment signals and is asked only to synthesise and personalise them — a narrower and considerably more reliable task.

2.1.4 Theme 4: Commercial Robo-Advisory Platforms

Commercial robo-advisors such as Betterment and Wealthfront represent the current state of the art in accessible, automated retail investment guidance. These platforms use Modern Portfolio Theory and factor-based allocation models to build diversified portfolios tuned to a user’s declared risk tolerance and investment horizon. They offer genuine personalisation at the portfolio level, but they operate exclusively on asset-class allocations — equities, bonds, ETFs — and produce no equity-level, directional recommendations with accompanying rationale. Nor do they incorporate real-time market signals, news sentiment, or LLM-generated explanations; their recommendations are typically static between rebalancing periods.

Trade Ideas is a commercial AI-driven stock screener that generates intraday trading signals using pattern recognition and machine learning. It targets professional day traders, costs several hundred dollars a month, provides no plain-language rationale for non-expert users, and has no personalisation against individual risk profiles.

2.2 Comparison of Existing Approaches

Table 2.1 summarises the key methodological properties and limitations of the reviewed works and platforms in relation to what QuantWise is required to deliver.

Table 2.1: Comparison of existing approaches against QuantWise requirements.

Reference / System	Approach	Strengths	Limitations / Gap
Fischer & Krauss (2018) [1]	LSTM on S&P 500 OHLCV data	Strong directional accuracy; outperforms memory-free baselines	No sentiment integration; no personalisation; diminishing returns post-2010
Bollen et al. (2011) [3]	Twitter mood as market predictor	Demonstrated measurable sentiment-return link	Market-level only; no individual stock or risk-profile targeting
Araci – FinBERT (2019) [4]	Domain-adapted BERT for financial sentiment	State-of-the-art financial NLP classification	Sentiment only; no quantitative prediction or advisory output
Lopez-Lira & Tang (2023) [7]	GPT-4 for news-to-return forecasting	Shows LLMs can capture market signals from text	Ungrounded generation; no risk profile; no quantitative signal integration
BloombergGPT (2023) [6]	Finance-specific 50B LLM	Superior on financial NLP benchmarks	Proprietary; no retail advisory; no personalisation or prediction pipeline
Betterment / Wealthfront	MPT-based robo-advisory	Genuine risk-profile personalisation; regulated	Asset-class level only; no equity signals; no LLM rationale; static
Trade Ideas	AI stock screener for traders	Real-time signals; broad ticker coverage	Professional audience; no plain-language rationale; no risk personalisation

Reference / System	Approach	Strengths	Limitations / Gap
QuantWise (this work)	LSTM + XG-Boost + FinBERT + LLM personalisation	End-to-end pipeline; equity-level signals; risk-personalised; explainable	Academic prototype; US equities only; daily cadence

2.3 Research Gap

The review reveals a clear and consistent gap: no existing publicly accessible system or published approach simultaneously satisfies all three requirements QuantWise is designed to meet.

- **Quantitative prediction systems** (e.g., Fischer & Krauss) produce directional signals but lack sentiment integration, risk grading, and any mechanism for personalising outputs to the individual investor.
- **Sentiment-augmented systems** improve signal reliability by incorporating news or social media data, but they do not produce equity-level, personalised, or explainable advisory recommendations.
- **LLM-based approaches** (e.g., Lopez-Lira & Tang) demonstrate that generative models can encode market-relevant information, but they operate in an ungrounded generative mode that is unreliable for personalised advisory use and carries significant hallucination risk.
- **Commercial robo-advisors** provide personalisation but at the asset-class level only, with no real-time market signals, no LLM-generated rationale, and no per-stock directional recommendations.

The specific gap QuantWise fills is therefore: *a unified, end-to-end platform that grounds LLM personalisation in validated quantitative and sentiment signals to produce equity-level, BUY/SELL/HOLD recommendations with plain-language rationale, adapted to the individual user’s risk profile, and delivered through an accessible web application.*

2.4 Theoretical Background

This section provides the theoretical background for the key technical components of QuantWise.

2.4.1 Long Short-Term Memory Networks

Long Short-Term Memory (LSTM) networks, introduced by Hochreiter and Schmidhuber [8], are a class of recurrent neural network designed to learn long-range temporal dependencies by maintaining a *cell state* C_t that is selectively updated through learnable gating mechanisms.

At each time step t , the LSTM computes:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (\text{forget gate}) \quad (2.1)$$

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (\text{input gate}) \quad (2.2)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \quad (\text{candidate cell}) \quad (2.3)$$

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \quad (\text{cell state update}) \quad (2.4)$$

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (\text{output gate}) \quad (2.5)$$

$$h_t = o_t \odot \tanh(C_t) \quad (\text{hidden state output}) \quad (2.6)$$

where x_t is the input at time t , h_{t-1} is the previous hidden state, σ is the sigmoid activation function, $\odot$ denotes element-wise multiplication, and $W_{(\cdot)}$, $b_{(\cdot)}$ are learned weight matrices and bias vectors. In QuantWise, the LSTM backbone processes a 60-day sliding window of scaled OHLCV and derived feature vectors, and its final hidden state h_T is passed to an XGBoost head for regression.

2.4.2 Gradient-Boosted Trees (XGBoost)

XGBoost [9] is a scalable, regularised gradient tree boosting framework that builds an ensemble of decision trees additively, with each new tree trained to correct the residual errors of the ensemble so far. The prediction at step t is:

$$\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + f_t(x_i) \quad (2.7)$$

where f_t is a newly fitted regression tree and x_i is the feature vector for sample i . In QuantWise, the XGBoost head receives the concatenation of the LSTM's final hidden state and 14 scaled technical indicator features, producing a normalised return prediction z that is denormalised using pre-computed training statistics to yield the estimated next-day percentage price change.

2.4.3 Transformer Architecture and FinBERT

The BERT (Bidirectional Encoder Representations from Transformers) architecture [5] uses a stack of self-attention layers to produce context-sensitive token embeddings by attending simultaneously to left and right context. Pre-training on large corpora lets BERT-derived models be fine-tuned effectively on downstream tasks with limited

labelled data.

FinBERT [4] extends BERT by further pre-training on a corpus of financial communications, giving it embeddings that better capture domain-specific terminology — analyst ratings, price target language, risk disclosures, and the like. In QuantWise, FinBERT classifies each news headline as positive, negative, or neutral, and those headline-level scores are averaged across all relevant headlines to form the per-ticker news sentiment component.

2.4.4 Command/Query Responsibility Segregation (CQRS)

CQRS is an architectural pattern that separates the read (query) and write (command) responsibilities of a software system into distinct models. Commands mutate state and report back only success or failure; queries read state and return data without side effects. In QuantWise, all backend operations are dispatched through the MediatR library, which routes each request to the appropriate handler. The separation makes handlers easy to unit-test in isolation, and it leaves the read model free to be optimised later without touching the command pipeline.

2.4.5 Transactional Outbox and Inbox Patterns

The Outbox pattern solves a classic distributed-systems problem: ensuring that a database state change and a corresponding message publication to a broker happen atomically — the dual-write problem. Rather than publishing directly to the broker, an event is written first to an `OutboxMessages` table within the same database transaction as the domain write. A background processor then reads and publishes unprocessed outbox messages to RabbitMQ, guaranteeing at-least-once delivery even when the broker is temporarily unavailable.

The complementary Inbox pattern handles idempotent consumption on the other side: received messages are recorded in an `InboxMessages` table before processing, which prevents duplicate handling if a message is redelivered. QuantWise implements both patterns for cross-module event propagation — specifically for the `UserCreated` domain event that triggers the welcome notification flow.

2.4.6 Modular Monolith Architecture

A modular monolith is a software architecture where a single deployable application is divided internally into well-defined, loosely coupled vertical modules, each with its own domain, application, infrastructure, and database context. Unlike a distributed microservices system, all modules share one process and deployment unit, eliminating the network latency, service discovery, and distributed tracing overhead that microservices bring. But unlike a traditional monolith, each module enforces strict boundary rules: cross-module communication happens only through published public API inter-

faces, never through direct database or service-layer references. QuantWise uses this pattern to house four modules: Users, Portfolio, Recommendations, and Notifications.

2.5 Datasets & Technologies

This section evaluates the external data sources, APIs, and engineering frameworks chosen for the QuantWise implementation.

2.5.1 Market Data: yFinance and Yahoo Finance

yFinance is an open-source Python library providing programmatic access to Yahoo Finance’s public market data API, returning historical OHLCV price data, analyst recommendations, price targets, and recent news for a wide range of equities. We chose it as the primary price data source because of its zero licensing cost, broad US large-cap coverage, and support for a dynamic ticker screener (`EquityQuery`) that lets the pipeline resolve the top 100 equities by market capitalisation at runtime. Rate limiting and proxy support are handled explicitly to avoid IP-level throttling.

2.5.2 Analyst and News Data: Finnhub API

Finnhub is a financial data platform offering REST API access to analyst consensus ratings, firm-level upgrade and downgrade histories, analyst price targets, and company-specific news headlines. QuantWise uses Finnhub as its primary source for analyst consensus data, falling back to Yahoo Finance when needed. For sentiment, Finnhub’s company-news endpoint returns up to 150 recent headlines per ticker per two-week window; these are filtered for company relevance before being passed to FinBERT for classification. The free tier caps requests at 60 API calls per minute, which the pipeline manages through a thread-safe throttle mechanism.

2.5.3 Backend Framework: ASP.NET Core and MediatR

ASP.NET Core is Microsoft’s open-source, cross-platform web framework. We chose it for the QuantWise backend because of its strong dependency injection support, minimal API routing, Entity Framework Core ORM integration, and a well-documented ecosystem for CQRS and clean-architecture patterns. MediatR is a lightweight .NET library that implements the mediator pattern for in-process command and query dispatching, enforcing clean separation between the API presentation layer and the application logic handlers.

2.5.4 Machine Learning Runtime: PyTorch and XGBoost

The LSTM backbone is implemented in PyTorch, the widely adopted deep learning framework that provides dynamic computation graphs, native GPU acceleration, and the `nn.LSTM` module used in the QuantWise architecture. The `mkldnn` backend is

explicitly disabled in the pipeline configuration to ensure thread-safe inference when multiple concurrent requests hit the FastAPI service. XGBoost provides the gradient-boosted regression head, loading a pre-trained model from disk at service startup.

2.5.5 Generative AI: Google Gemini API

Google Gemini is the multimodal large language model API used in QuantWise at request time to personalise the market-wide prediction set against each user's risk profile. The API is called with a schema-constrained prompt that forces the response into a validated JSON structure (`{summary, picks[]}`), and the pipeline retries up to three times on a parse failure before surfacing an error to the caller. The 24-hour Redis cache on the response minimises both API cost and latency for subsequent requests within the same trading session.

2.5.6 Infrastructure: PostgreSQL, Redis, and RabbitMQ

PostgreSQL 18 serves as the primary relational data store, with one EF Core database context per module maintaining strict schema isolation. Redis acts as a hybrid cache layer (in-memory plus distributed) for the Gemini-generated recommendation payloads, configured with a 24-hour time-to-live aligned with the daily pipeline cadence. RabbitMQ is the message broker for asynchronous cross-module event delivery, configured with MassTransit to abstract transport-level concerns and provide automatic retry and dead-letter queue support.

Chapter 3

Software Requirements & Project Management

Two questions run through this chapter. What must QuantWise do, and how did two people actually get it built? The Software Requirements Specification answers the first. It names the people the system serves, states the functional and non-functional requirements, traces them back to the use cases and user stories they came from, and sets out the constraints the design had to live with. None of the requirements are left vague — each is written to be checked, and a traceability matrix toward the end ties every one to the component that delivers it and the test that proves it works. The rest of the chapter answers the second question: how the work was organised, how progress was tracked, and which risks actually bent the plan while the project was live.

3.1 Stakeholders & Users

The stakeholder list is short, as tends to be the case for a student project, but the parties involved are not interchangeable — each one shaped something about the build.

3.1.1 Organisational Stakeholders

Dr. Marwa Solayman, the academic supervisor, sets the academic scope and judges how well the system is engineered. Eng. Farah Darwish, the co-supervisor, advises on the technical side and reviews progress between milestones. The development team is the two authors. Between them they owned every requirement, every line of code, and the calendar. A team this small has no separate analyst or QA group, so the developers are the main internal stakeholder as well. The examination committee assesses the finished dissertation and the defence, and it makes one demand that never changes. Every claim has to be backed by something in the repository or the test results.

3.1.2 End-User Personas

Two personas drove the design, since the product is meant for ordinary users. The first is the first-time investor, a young professional with a self-directed brokerage account, not much financial background, and very little time to research stocks. This user wants

a few clear suggestions sized to the risk they can live with, and wants a reason attached before acting. The second is the hands-on hobbyist, who already holds a handful of positions and uses QuantWise for a second opinion. This user mostly cares whether today’s signals fit the positions they already hold. The recommendation engine reasons about a user’s existing holdings for exactly that reason, so a returning investor is given hold and sell guidance on what they own.

3.1.3 System Actors

Three non-human actors take part too, and they earn a mention because some requirements answer to them rather than to a person. The Daily Scheduler is the in-process Quartz job that pulls the pipeline’s output once a day. The Prediction Pipeline is the Python service that turns market data into risk-graded predictions. The Generative AI Provider is the Gemini API, which adapts those market-wide signals to one user at the moment of the request. Table 3.1 sets out how the human parties touch the platform.

Table 3.1: Stakeholder and user interactions with the platform.

Party	Role	Primary Interactions
First-time investor	Primary user	Registers, completes the risk questionnaire, reads personalised recommendations and their dollar allocations, checks notifications.
Hands-on hobbyist	Secondary user	Everything the primary user does, plus reviews hold and sell guidance against current holdings and explores the market and learning views.
Supervisor and TA	Sponsor and assessor	Review milestones and demonstrations, evaluate engineering quality, guide scope.
Examination committee	Assessor	Assess the final artefact and the defence against the stated requirements.

3.2 Functional Requirements

Requirements are grouped by module and ranked with MoSCoW (Must, Should, Could, and Won't-have). Each is worded so a single test can settle whether it passes, and Section 3.6 records that test. Table 3.2 lists them.

Table 3.2: Functional requirements with MoSCoW prioritisation.

Req ID	Module	Functional Requirement	Priority
FR-01	Users / Auth	The system shall let a visitor register with first name, last name, email, and password. The password shall be stored only as a BCrypt hash, and a duplicate email shall be rejected with a clear message.	Must
FR-02	Users / Auth	The system shall authenticate a registered user from email and password and return a signed JWT bearer token (HMAC-SHA256) on success. Invalid credentials shall be rejected without revealing which field was wrong.	Must
FR-03	Users / Auth	An authenticated user shall be able to retrieve their own profile: identifier, name, email, and role.	Must
FR-04	Users / Auth	Every non-public endpoint shall require a valid JWT, and a user shall be able to read only their own user and portfolio resources. Reading another user's resource shall return HTTP 403.	Must
FR-05	Portfolio	The system shall present a risk questionnaire covering primary goal, time horizon, risk tolerance, market-reaction attitude, and investment experience, and derive a risk profile of Conservative, Moderate, or Aggressive together with a target asset allocation.	Must

Req ID	Module	Functional Requirement	Priority
FR-06	Portfolio	The system shall persist the questionnaire outcome as a portfolio: the risk profile and the stocks, bonds, ETFs, and cash percentages, linked to the owning user.	Must
FR-07	Portfolio	A user shall be able to retake the questionnaire and update the existing portfolio in place (HTTP PUT), without creating a second portfolio.	Must
FR-08	Portfolio	A user shall be able to view their own portfolio, including the risk profile and the allocation breakdown.	Must
FR-09	Portfolio	A user shall be able to record the amount of capital they intend to invest, and the system shall turn allocation percentages into per-pick dollar amounts.	Should
FR-10	Recommendation	The system shall generate a ranked set of BUY, SELL, and HOLD recommendations personalised to the user's risk profile from the latest daily run. Each pick shall carry an allocation percentage, a one-sentence reason, and a one-sentence risk note, alongside an overall summary.	Must
FR-11	Recommendation	Recommendation generation shall account for the user's current holdings, returning HOLD or SELL for tickers already held rather than treating each request as a fresh portfolio.	Should
FR-12	Recommendation	Generated recommendations shall be cached per user for 24 hours and served from cache on later requests in that window, so that at most one model generation happens per user per day.	Must

Req ID	Module	Functional Requirement	Priority
FR-13	Recommendations	The system shall expose the latest market-wide daily predictions for the learning view, without letting the user pick an arbitrary past date.	Should
FR-14	Recommendations	A scheduled job shall fetch the pipeline's daily output and ingest it as a dated run. Ingestion shall be idempotent on the run's generation timestamp, so a repeated fetch does not create a duplicate run.	Must
FR-15	Recommendations	The system shall expose a secured internal endpoint that accepts a daily results payload authenticated by an API key, so the pipeline output can be ingested out of band.	Should
FR-16	Market	The system shall provide live quotes and symbol search to authenticated users by proxying a market-data provider, keeping the provider key on the server.	Should
FR-17	Notifications	On successful registration the system shall raise a welcome notification for the new user.	Should
FR-18	Notifications	After each daily run is ingested, the system shall create a personalised "new recommendations are ready" notification for each eligible user.	Should
FR-19	Notifications	A user shall be able to list their notifications and read an unread count.	Must
FR-20	Notifications	A user shall be able to mark a single notification, or all of them, as read.	Must

Req ID	Module	Functional Requirement	Priority
FR-21	Pipeline	The pipeline shall produce, for the top US large-capitalisation equities, a next-day directional prediction (UP or DOWN) with an estimated percentage change and a confidence score, using the LSTM and XG-Boost ensemble.	Must
FR-22	Pipeline	The pipeline shall compute a per-ticker composite sentiment signal from FinBERT-classified news headlines and analyst consensus data.	Must
FR-23	Pipeline	The pipeline shall apply deterministic risk rules that assign each equity a risk level (LOW, MEDIUM, HIGH), a conviction score, a trading signal, and any applicable risk flags, so the same inputs always produce the same grading.	Must
FR-24	Data Export	The platform may let a user export their recommendation set to a portable format for offline reference.	Won't (this iteration)

FR-24 appears in the table even though it is out of scope for this iteration. Including it keeps the boundary between what was built now and what comes next explicit. Chapter 6 picks it up under future work.

3.3 Non-Functional Requirements (NFR)

Where the functional requirements say what the system does, the non-functional ones set the quality bar. Every entry carries a measurable target that Chapter 5 can verify. A few reflect design goals for a single-server prototype, and the criterion says so plainly where a figure has not been confirmed under production load. Table 3.3 lists them.

Table 3.3: Non-functional requirements with measurable targets.

NFR ID	Quality Attribute	Measurable Target Criterion
NFR-01	Performance (reads)	Authenticated reads of profile, portfolio, and notifications return in under 300 ms on the reference single-server deployment under light concurrency, and a cache-hit recommendation request returns in under 200 ms.
NFR-02	Performance (AI path)	A cache-miss recommendation request, which calls Gemini, completes in under 10 s, and the 24-hour cache (FR-12) keeps that cost to at most once per user per day.
NFR-03	Security (authentication)	Passwords are never stored or logged in plaintext; only BCrypt hashes are persisted. JWTs are signed with an HMAC-SHA256 key of at least 256 bits held outside source control.
NFR-04	Security (authorization)	Resource ownership is enforced on all user and portfolio reads, so no authenticated user can reach another user's data. This is confirmed by a negative test that expects HTTP 403.
NFR-05	Security (secrets, transport)	All connection strings, API keys, and signing keys are supplied from the environment and excluded from version control. The API is deployable behind TLS, and cross-origin access is restricted to known origins in a production configuration.
NFR-06	Reliability (messaging)	Cross-module events are delivered at least once through the transactional outbox and inbox, consumers are idempotent, and failed messages are retried and dead-lettered rather than lost.

NFR ID	Quality Attribute	Measurable Target Criterion
NFR-07	Availability (fault tolerance)	A failed daily pipeline fetch does not crash the application; it is logged and retried on the next cycle. A malformed LLM response is retried up to three times before the request surfaces an error.
NFR-08	Maintainability (modularity)	Cross-module communication happens only through published public-API interfaces, each module owns its own database context, and CQRS handlers are unit-testable in isolation. The bar is a green automated suite of at least 50 tests.
NFR-09	Usability and accessibility	The web interface is responsive, honours the prefers-reduced-motion setting, shows a visible keyboard focus ring, and renders explicit loading, empty, and error states for every asynchronous view.
NFR-10	Portability	The full stack (database, cache, broker, mail) is reproducible from a single Docker Compose file and brought up with one command.
NFR-11	Scalability	The API is stateless and the cache is distributed, which allows horizontal scaling in a future deployment, and the pipeline is decoupled as a once-a-day batch so it never sits on the request path.
NFR-12	Cost-efficiency	Generative-AI spend is bounded by the per-user daily cache, and market and news data are sourced from free API tiers behind a rate throttle.

3.4 Use Cases & User Stories

Each requirement above traces back to a use case that describes how an actor reaches a goal. The actors are the visitor before logging in, the authenticated investor (covering both personas), the Daily Scheduler, the Prediction Pipeline, and Gemini. Figure 3.1

shows the use-case diagram. Four of the central use cases are written out in full below, including flows and acceptance criteria, and the rest are summarised.

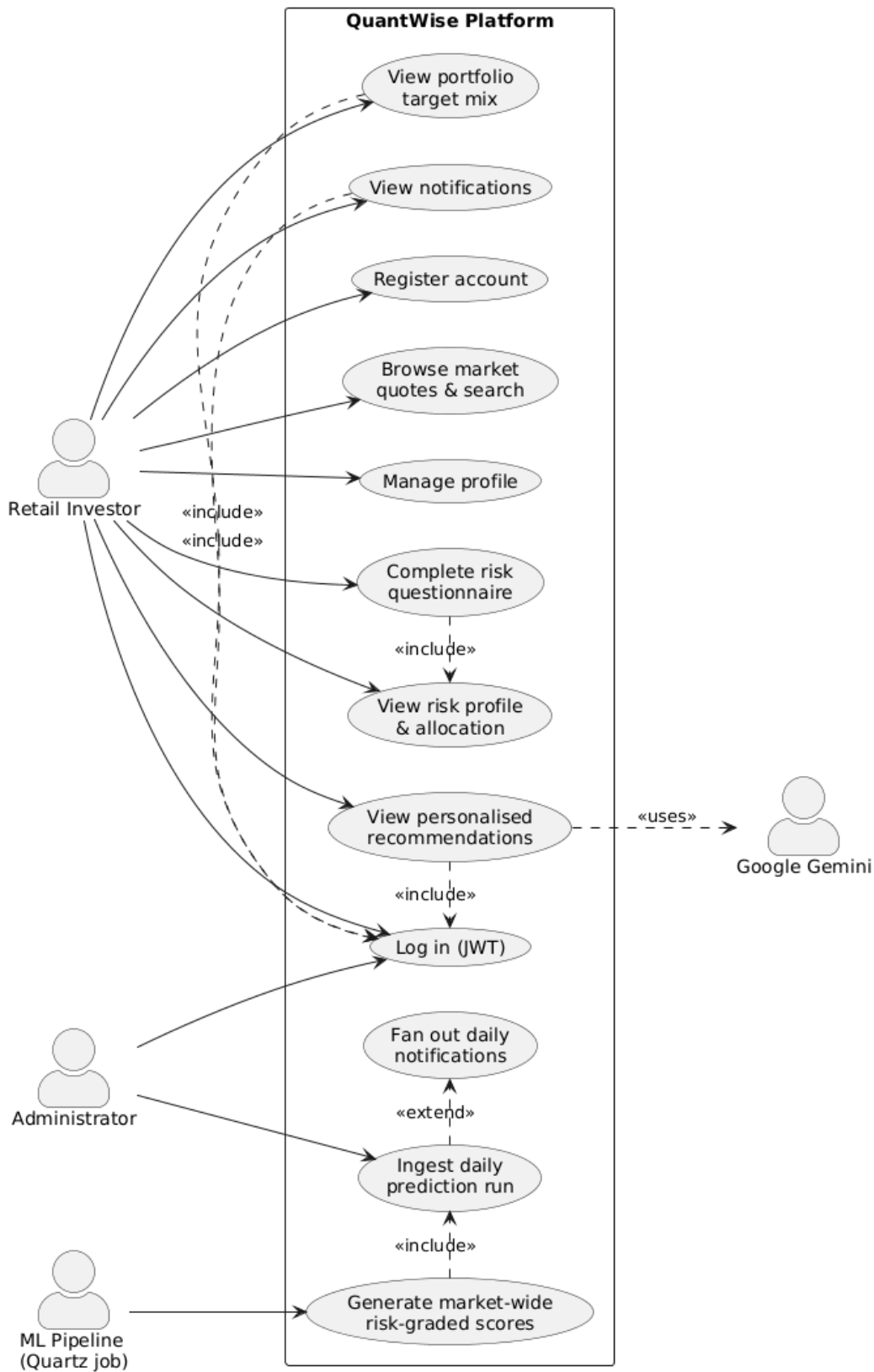


Figure 3.1: QuantWise use-case diagram.

3.4.1 Detailed Use Cases

Table 3.4: UC-02 — Authenticate (Log In).

Field	Description
Use Case ID	UC-02
Actor	Investor (registered)
Goal	Obtain a session token and reach the right landing page.
Preconditions	The user has a registered account.
Main Flow	1. The user submits email and password. 2. The system finds the user by email. 3. The system verifies the password against the stored BCrypt hash. 4. The system issues a JWT signed with HMAC-SHA256. 5. The frontend stores the token and checks whether a portfolio exists. 6. The user is routed to the dashboard.
Alternative Flow	5a. No portfolio exists, so the user is routed to onboarding instead.
Exception Flow	3a. The credentials are invalid: the system returns HTTP 401 with a generic message and issues no token.
Postconditions	An authenticated session exists, and later requests carry the bearer token.
Acceptance Criteria	Valid credentials yield a token and the dashboard within 2 seconds. Invalid credentials yield 401 without disclosing which field failed. Protected routes reject requests that carry no valid token (FR-02, FR-04).

Table 3.5: UC-03 — Complete the Risk Questionnaire.

Field	Description
Use Case ID	UC-03
Actor	Investor
Goal	Produce a risk-graded portfolio that later personalises recommendations.

Field	Description
Preconditions	The user is authenticated. They either have no portfolio yet or are retaking the questionnaire.
Main Flow	1. The user answers goal, horizon, risk tolerance, market reaction, and experience, and enters an investment amount. 2. The client computes a risk score and maps it to a profile and an asset allocation. 3. The user submits. 4. The backend persists the portfolio and links it to the user. 5. The user is redirected to the dashboard.
Alternative Flow	1a. Retake: the existing answers are prefilled, and submission updates the portfolio through HTTP PUT rather than creating a new one.
Exception Flow	3a. Validation fails: inline errors are shown and nothing is submitted.
Postconditions	A portfolio is stored, and the recommendation engine can personalise against it.
Acceptance Criteria	The derived profile is one of Conservative, Moderate, or Aggressive. The allocation percentages sum to 100. A re-take updates the existing portfolio instead of duplicating it (FR-05, FR-06, FR-07, FR-09).

Table 3.6: UC-04 — View Personalised Recommendations.

Field	Description
Use Case ID	UC-04
Actor	Investor (Gemini and the latest daily run are supporting actors)
Goal	See a ranked, justified set of picks matched to the user's risk profile.
Preconditions	The user is authenticated, a portfolio exists, and at least one daily run has been ingested.

Field	Description
Main Flow	1. The user opens the dashboard. 2. The backend checks the Redis cache for this user. 3. On a miss, it loads the latest run, the user's risk profile, and the user's holdings. 4. It builds a schema-constrained prompt and calls Gemini. 5. It parses the returned JSON into a summary and a list of picks. 6. It caches the result for 24 hours. 7. It returns BUY, SELL, and HOLD picks with allocation, reason, and risk note. 8. The frontend renders the picks and computes dollar amounts from the investment amount.
Alternative Flow	2a. Cache hit: the cached recommendation set is returned without calling Gemini.
Exception Flow	3a. No daily run exists: the dashboard shows an empty state. 4a. The model reply will not parse: the call is retried up to three times before an error is surfaced.
Postconditions	The recommendations are displayed, and the user's holdings are updated from the accepted picks.
Acceptance Criteria	Picks reflect the user's risk profile, held tickers appear as HOLD or SELL, and a second request inside 24 hours is served from cache (FR-10, FR-11, FR-12).

Table 3.7: UC-07 — Ingest the Daily Run and Notify Users (system).

Field	Description
Use Case ID	UC-07
Actor	Daily Scheduler (Quartz), with the Prediction Pipeline as a supporting actor
Goal	Keep the platform's signals fresh and tell users when new picks are available.
Preconditions	The pipeline service is reachable, and the backend is running.

Field	Description
Main Flow	1. The scheduler fires on its cron trigger (01:00 UTC, Tuesday to Saturday). 2. The job calls the pipeline's scoring endpoint. 3. It dispatches an ingest command with the returned run. 4. The run is stored as a dated record, keyed on its generation timestamp. 5. A domain event triggers a personalised notification for each eligible user.
Alternative Flow	4a. A run with the same timestamp already exists, so the existing run is returned and nothing is duplicated.
Exception Flow	2a. The pipeline is unreachable: the failure is logged and the job returns quietly, waiting for the next trigger.
Postconditions	A fresh run is stored and users have been notified.
Acceptance Criteria	Ingestion is idempotent on the timestamp, a pipeline failure never crashes the app, and one notification per eligible user is created per new run (FR-14, FR-18, NFR-07).

The remaining use cases share the same structure and appear in summary form in Table 3.8.

Table 3.8: Summary of the remaining use cases.

UC ID	Name	Actor	Goal
UC-01	Register	Visitor	Create an account with a hashed password.
UC-05	Manage portfolio	Investor	Edit the portfolio and set the investment amount.
UC-06	Manage notifications	Investor	Read notifications and mark them as read.
UC-08	Browse market data	Investor	Look up live quotes and search for symbols.

3.4.2 Representative User Stories

During development, these use cases were expressed as user stories, each with a concrete acceptance test already in mind. Four of them stand in for the rest below.

- **US-1.** As a first-time investor, I want a short risk questionnaire so the app understands how much risk I can take. *Accepted when* the answers map to one of three

profiles and produce an allocation that sums to 100%.

- **US-2.** As an investor, I want each recommendation to tell me why it was made so I can act with the right amount of confidence. *Accepted when* every pick carries a one-sentence reason and a one-sentence risk note.
- **US-3.** As an investor who already owns stocks, I want hold-or-sell guidance so I am not told to re-buy what I hold. *Accepted when* tickers I already hold appear only as HOLD or SELL.
- **US-4.** As a returning user, I want to know when fresh recommendations are ready. *Accepted when* a notification appears after each daily run is ingested.

3.5 System Constraints & Assumptions

A handful of constraints shaped the design, and naming them explains decisions that otherwise look arbitrary. The market-data sources are free tiers with firm rate ceilings. Finnhub’s free plan caps requests at 60 per minute, which is why the pipeline routes its news and analyst calls through a single thread-safe throttle, and why Finnhub is called from the backend to keep the API key off the client. The equity universe is limited to the top US large-cap names, matching the model the team trained. The pipeline runs once a day, so the guidance covers the next day only. The entire stack is designed to run on one machine under Docker Compose, with no Kubernetes and no multi-region deployment. LSTM inference is pinned to the thread-safe native RNN path because the oneDNN kernel becomes non-deterministic off the main thread — and the pipeline configuration enforces that explicitly.

The assumptions are equally concrete. Training happens offline, and the models are loaded from files at pipeline start-up, so a running instance only infers and never trains. Gemini is assumed reachable at request time, with the 24-hour cache and the three-attempt retry absorbing the occasional failure. Users are assumed to be on a modern desktop browser. The platform is a prototype for decision support in an academic setting: it gives guidance, never executes a trade, and makes no claim to comply with financial-advisory regulation.

3.6 Requirements Traceability

Traceability is what keeps requirements anchored to the code. Table 3.9 ties each functional requirement to the use case behind it, the component that implements it, and how it gets verified. That last column points ahead to Chapter 5, where the 68-test xUnit suite and the black-box cases close the loop.

Table 3.9: Requirements traceability matrix.

Req	Use Case	Implementing Component	Verification
FR-01	UC-01	Users module, RegisterUser, PasswordHasher	Unit test + black-box TC
FR-02	UC-02	Users module, LoginUser, JwtTokenGenerator	Unit test + black-box TC
FR-03	UC-02	Users module, GetUserProfile	Black-box TC
FR-04	UC-02,05	JWT middleware, ownership checks on user and portfolio reads	Negative test (403)
FR-05, FR-06	UC-03	Portfolio module, CreatePortfolio, onboarding flow	Domain unit test
FR-07	UC-03,05	Portfolio module, UpdatePortfolio (PUT)	Domain unit test + TC
FR-08	UC-05	Portfolio module, GetPortfolioByUserId	Black-box TC
FR-09	UC-03,05	Portfolio InvestmentAmount, dashboard dollar split	Domain unit test
FR-10, FR-11	UC-04	Recommendations, GetRecommendations, GeminiLlmClient, holdings	Handler test + TC
FR-12	UC-04	Redis HybridCache, 24-hour entry	TC (cache hit)
FR-13	UC-04	Recommendations, GetLatestPredictions	Handler test
FR-14	UC-07	FetchDailyPipelineJob, IngestDailyRun (idempotent)	Handler test

Req	Use Case	Implementing Component	Verification
FR-15	UC-07	Internal daily-results endpoint, API-key auth	Black-box TC
FR-16	UC-08	Market endpoints, Finnhub proxy	Black-box TC
FR-17, FR-18	UC-07	Notifications consumers, daily-run event handler	Handler test
FR-19, FR-20	UC-06	Notifications, query and mark-read handlers	Unit tests
FR-21, FR-22, FR-23	UC-07	Pipeline: LSTM+XGBoost, FinBERT, <code>risk_rules.py</code>	Pipeline output inspection

3.7 Project Management Methodology & Task Allocation

Two people built QuantWise, and the management story is best told plainly. The team did not follow a formal methodology with named ceremonies. There were no daily stand-ups and no sprint retrospectives, and with only two developers, no separate Scrum Master either — those rituals cost time without buying coordination that two people in constant contact already have. What the team did run, in practice, was a continuous-flow process along Kanban lines, with GitHub as the one board that mattered.

3.7.1 Why Kanban Rather Than Scrum

The choice was deliberate, so the alternatives are worth addressing. Scrum organises work into fixed sprints with defined roles and a standing set of ceremonies, and it is tuned for teams of about five to nine people. Shrink that to two and the roles collapse into each other while the ceremonies become theatre. Kanban asks for none of that: items cross a simple board from backlog to in-progress to done, and the team controls how much is open at any one time. That matched how the project actually moved — in focused bursts of delivery against clear goals that show up directly in the commit and pull-request history. Scrumban came up as an option, but it pays off only when a team does real up-front iteration planning, which this one did not. Kanban is the most accurate label for what happened, and the easiest to defend.

3.7.2 Project Scope

Chapter 1 sets out the full scope, so this is just a reminder. The team committed to an end-to-end decision-support platform: an offline-trained prediction pipeline, a modular-monolith backend, and a React frontend, all aimed at personalised BUY, SELL, and HOLD calls for the top US large-cap equities. Trade execution, non-US assets, intraday signals, and native mobile apps were deliberately excluded to keep the project inside what two people could actually finish.

3.7.3 Team Roles and Work Breakdown

Rather than fixed titles, the two developers worked in lanes that suited their strengths. One focused on the machine-learning pipeline and the backend platform, the other on the frontend, integrations, and quality work, and both shared the architecture and design decisions. The per-person detail needed for individual marking lives in the Individual Contributions chapter. Table 3.10 breaks the work down by epic.

Table 3.10: Work breakdown structure by epic.

Epic	Area	Key Deliverables	Primary Area
E1	ML pipeline and models	Data acquisition, feature engineering, LSTM+XGBoost training, FinBERT sentiment, risk rules, FastAPI service	ML
E2	Backend platform	Modular-monolith skeleton, common layers, per-module EF Core contexts	Backend
E3	Users and auth	Register, login, JWT, BCrypt, profile	Backend
E4	Portfolio and onboarding	Questionnaire, risk profile, allocation, investment amount	Backend
E5	Recommendations	Daily ingest, Gemini personalisation, holdings, caching, predictions	Backend
E6	Notifications	Welcome and daily-run notifications, read state	Backend
E7	Messaging backbone	Outbox/inbox, RabbitMQ and MassTransit, Quartz scheduling	Backend

Epic	Area	Key Deliverables	Primary Area
E8	Frontend	React and TypeScript, TanStack Query, the Quant Terminal design, all pages	Frontend
E9	Cross-cutting	Security hardening, secret management, market proxy	Shared
E10	Testing and QA	The 55-test xUnit suite across six projects	Shared
E11	Infrastructure	Docker Compose for Postgres, Redis, RabbitMQ, and Mailpit	Shared

3.8 Schedule & Progress Tracking

The schedule here is rebuilt from the project’s own GitHub history — the most reliable record of when things actually happened. Work ran from February to June 2026, unevenly. The first serious effort went into the offline model research and training that produced the LSTM backbone, the XGBoost head, and the scalers the pipeline loads at start-up. That work leaves almost no trace in the application repository. The only artefacts committed there were the finished model files; the training experiments themselves lived elsewhere. It also explains the quiet stretch in the commit log between late February and early June. Through the inter-semester period, the team’s time went to the model work and to coursework, and application development resumed in concentrated bursts once the models were done. Table 3.11 lists the milestones with their real dates.

Table 3.11: Project milestones reconstructed from the GitHub history.

ID	Milestone	Period	Status
M0	Model research and of-line training	Feb–May 2026	Completed
M1	Platform foundation: modular backend, auth, Portfolio and Notifications modules, initial React UI	9–25 Feb 2026	Completed

ID	Milestone	Period	Status
M2	Asynchronous backbone and the Recommendations module (outbox/inbox)	2–7 Jun 2026	Completed
M3	Pipeline service, model integration, and the Quant Terminal frontend rebuild	12–16 Jun 2026	Completed
M4	Integration, security hardening, and the test suite	16–17 Jun 2026	Completed

Every milestone aligns with the commit record. The foundation milestone (M1) laid down the modular skeleton, JWT authentication, the Portfolio and Notifications modules, and the first React pages, and it included the template-fix pull request (#1). Work resumed in June with the Recommendations module and the transactional outbox and inbox across Portfolio and Recommendations (M2). M3 was the heaviest stretch: it wrapped the trained models in the FastAPI pipeline, containerised the service, standardised API routes under `/api/*`, moved the signal vocabulary to BUY, SELL, and HOLD, and shipped the TypeScript frontend rebuild through pull request #2. The final push (M4) brought the investment-amount feature, the Finnhub market proxy, the real-data learning view, holdings-aware recommendations, and daily-run notifications. It also closed out the security and ownership fixes, moved secrets into the environment with a rotated JWT key, switched the cache to 24 hours, and grew the test suite to 68 passing tests. Figure 3.2 shows the same plan as a Gantt chart.

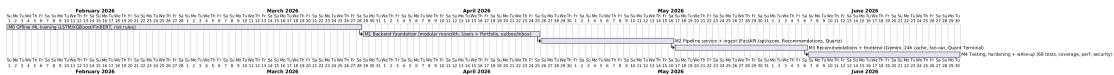


Figure 3.2: Project schedule as a Gantt chart.

3.8.1 Progress-Tracking Method

Tracking happened on GitHub, from start to finish. Issues held the work items, pull requests carried the reviews, and the commit history provided the running record of what got done and when. The two merged pull requests — #1 for the template fixes and #2 for the frontend rebuild — mark where the bigger chunks of work came together after review. Because the code and its full history are in one place, the schedule above could be reconstructed directly from that record.

3.9 Risk Management & Mitigation Strategy

Risk was managed continuously throughout the project, without a separate register that would have gone stale. The risks that mattered, and the response to each, are still straightforward to document. Two of them actually materialised, and Table 3.12 records how they were resolved — plainly and without spin.

Table 3.12: Risk register and mitigation.

Risk	Likelihood / Impact	Mitigation and Outcome
Market-data API throttling or deprecation (free-tier Finnhub and yfinance)	Medium / High	A thread-safe rate throttle in the pipeline, Finnhub as primary with Yahoo as a fallback, and a server-side proxy. Rate limits were hit during development and handled by throttling and caching.
Daily pipeline run failing silently and signals going stale	Medium / High	Materialised. An ingest bug stored the run timestamp with local kind, which the database rejected, so every run failed with a 500 and the latest data went stale. Fixed by normalising timestamps to UTC, after which a fresh run ingested cleanly.
Secrets committed to the repository	Low / High	Materialised. Keys and connection strings had been sitting in app settings. Moved to a git-ignored environment file loaded at start-up, the settings file was blanked, and the JWT signing key was rotated.
Generative-AI cost and latency	Medium / Medium	A 24-hour per-user cache caps generation to once per day, and a schema-constrained prompt with up to three retries keeps malformed replies from reaching the user.

Risk	Likelihood / Impact	Mitigation and Outcome
Model staleness or accuracy decay over time	Medium / Medium	Models are retrainable offline, daily inference keeps signals current, and the deterministic risk layer flags contradictions between price and sentiment.
Two-person bandwidth and scope creep	High / Medium	MoSCoW prioritisation, an explicit out-of-scope list (data export, mobile, intraday), and a low work-in-progress limit kept the effort focused.
The inter-semester scheduling gap	Medium / Medium	The gap was used deliberately for the offline model training, and application delivery was concentrated into focused June bursts once the models were ready.
Hidden coupling inside the monolith	Medium / Medium	Cross-module calls were restricted to published public-API interfaces and each module kept its own database context, which preserved module boundaries.

3.10 Communication Plan

Two developers do not need much communication machinery. The pair coordinated in person and online, running everything asynchronously through GitHub — issues tracked the work, pull-request reviews and commit messages left the written trail. Contact with the supervisor and teaching assistant came at milestone checkpoints, where progress was demonstrated and the feedback fed back into the plan. It is a modest setup, but it suits two people, and it produced the evidence trail this chapter has relied on.

Chapter 4

System Design and Implementation

This chapter takes the requirements from Chapter 3 and shows what was actually built. We start from the outside in: the three-tier architecture and how the tiers communicate, then dive into the backend, where most of the engineering weight sits. The prediction pipeline, UML models, database design, interface, and technology choices follow in turn. Every diagram reflects the system as it was built, and every code listing is drawn straight from the running codebase.

4.1 System Architecture Overview

QuantWise is a polyglot, three-tier system with one unifying goal: turning raw market data into personalised, risk-graded guidance that a non-expert can act on. Each tier uses the language best suited to its job. Machine learning lives in a Python service, domain logic and the user-facing API live in a .NET backend, and the client is a React application. Figure 4.1 shows how they fit together.

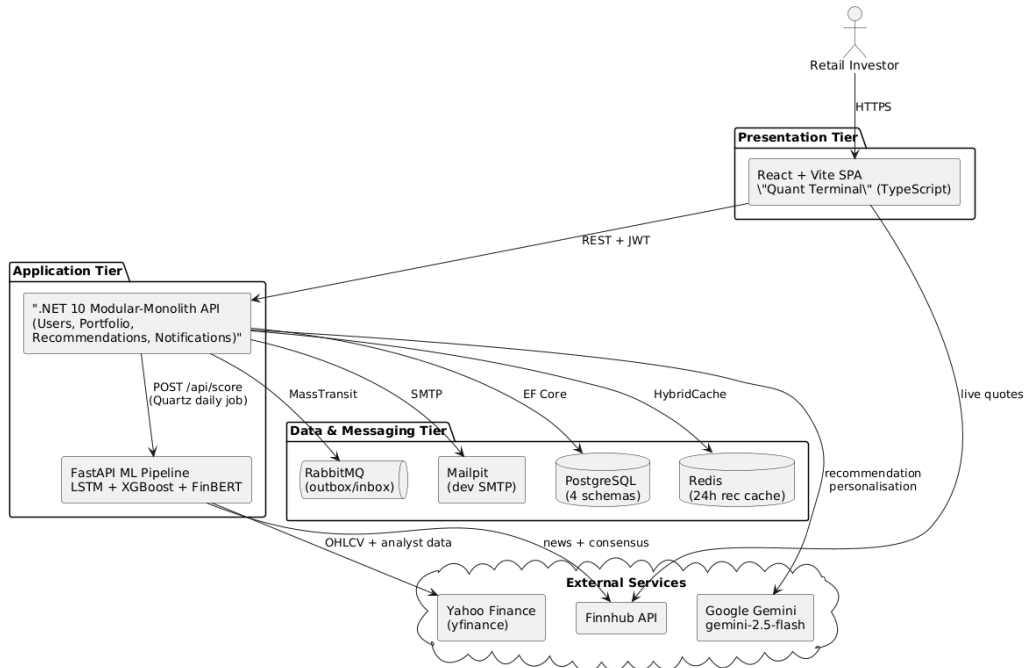


Figure 4.1: High-level system architecture of QuantWise.

Two flows run through that picture, and they run on different clocks. The first is a nightly batch: once a day, a scheduled job inside the backend calls the pipeline, which scores the market and returns a graded prediction set for the backend to store. The second is the live request path. When a user opens the dashboard, the backend takes that stored prediction set together with the user’s risk profile and current holdings and asks Google Gemini to condense everything into a short, personalised list of BUY, SELL, and HOLD calls. Splitting the work this way keeps the expensive machine learning off the request path, so a dashboard load involves only cheap reads and one cached model call.

The entire runtime starts from a single Docker Compose file. Postgres 18 holds relational data on port 5432, Redis 8 serves the cache on 6379, and RabbitMQ 4 handles messaging on 5672 with a management console on 15672. Mailpit captures outgoing mail in development, and the pipeline image listens on 8000 for the daily scoring call. Figure 4.2 shows the deployment topology.

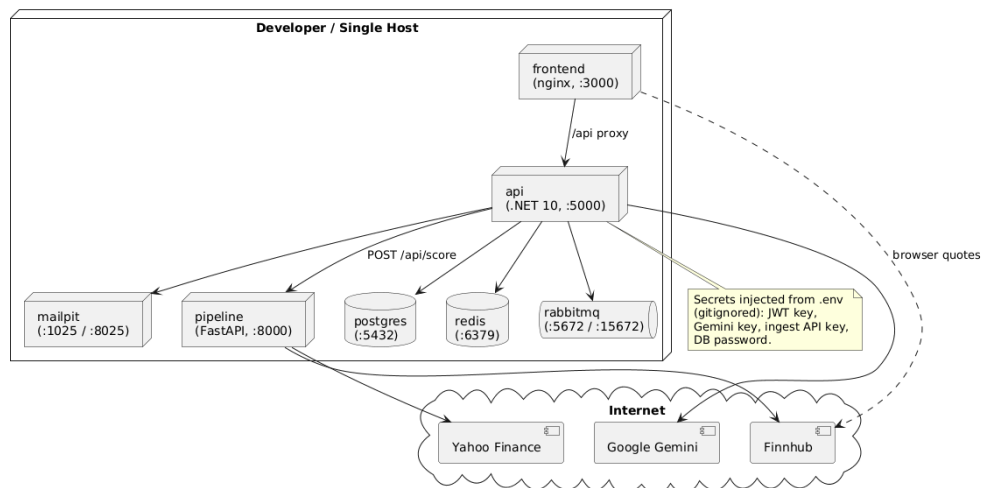


Figure 4.2: Deployment topology under Docker Compose.

4.2 Backend Architecture: the Modular Monolith

The backend is a single deployable application divided into four vertical modules: Users, Portfolio, Recommendations, and Notifications. Each module is a self-contained slice that owns its domain, its database context, and the small API surface it exposes to other modules. The four share one process and one deployment unit, keeping operations simple, yet they never share internals.

4.2.1 Layers and the Dependency Rule

Every module is built from the same six projects. Domain holds the entities and domain events, depending on nothing else. Application sits above it with the use-case logic, written as command and query handlers. Infrastructure supplies the concrete imple-

mentations — mostly EF Core persistence and external clients — while Presentation exposes the HTTP endpoints. Two thin projects, `IntegrationEvents` and `PublicApi`, hold the contracts that other modules may see. Dependencies always flow inward toward the domain, keeping business rules free of any database or web-framework concerns. A shared `Common` project provides the cross-cutting pieces every module needs: the messaging abstractions, the cache service, and the outbox and inbox plumbing. Figure 4.3 shows the module and layer structure.

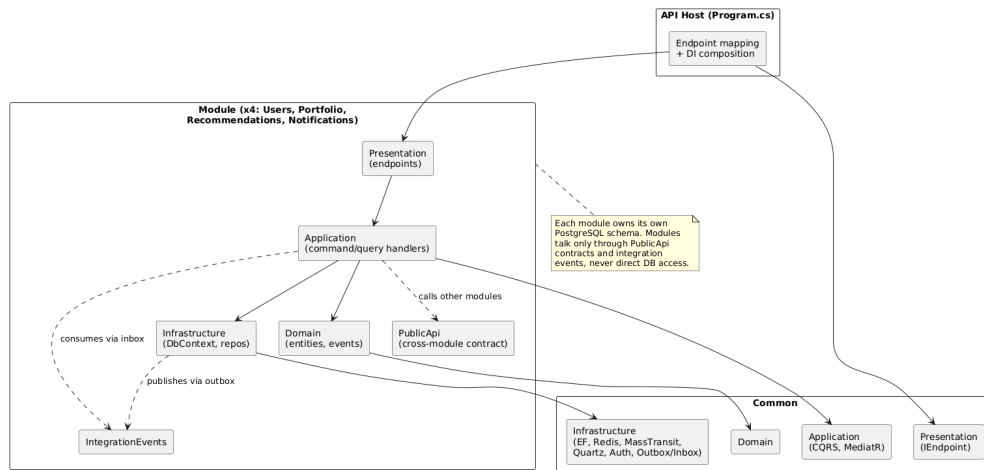


Figure 4.3: Backend module and layer structure.

4.2.2 CQRS with MediatR

Inside a module, every operation is either a command or a query, dispatched through MediatR. A command changes state and reports back only success or failure. A query reads state and returns data. Handlers implement `ICommandHandler` or `IQueryHandler` and return a `FluentResults Result`, so a failure travels back as a typed error with no exceptions thrown across layers. That separation makes the domain logic straightforward to test in isolation — and it is why the handlers account for most of the 68 unit tests. The Presentation endpoints stay thin: each one reads the request, sends a single command or query, and maps the `Result` to an HTTP response.

4.2.3 Module Boundaries

The boundary rule is strict. A module never reaches into another module’s database or calls its internal services directly. When Recommendations needs a user’s risk profile, it calls `IPortfolioApi` — a small published interface from the Portfolio module’s `PublicApi` project — and receives a flat response object back. Notifications reaches Users and Portfolio the same way. Nothing else crosses a boundary apart from these published contracts and the integration events described below. That discipline is what stops a monolith from sliding into a tangle of hidden dependencies, and it is the main reason the modular style earned its larger project count.

4.2.4 Asynchronous Messaging: Outbox, Inbox, and Quartz

Some operations must cross modules without making the user wait. Creating a user should trigger a welcome notification, and ingesting a daily run should notify everyone with a portfolio — yet neither action belongs in the request that started it. QuantWise solves this with the transactional outbox and inbox patterns over RabbitMQ, brokered by MassTransit.

When a handler saves changes, an EF Core interceptor gathers the domain events raised by entities in that transaction and writes them to an `outbox_messages` table in the same commit. The data change and the intent to publish therefore succeed or fail as one, closing the dual-write gap. A Quartz job in each module, `ProcessOutboxJob`, reads the unpublished rows and hands them to the broker, so a broker outage only delays delivery and never loses it. On the receiving side, a matching inbox job records each message before processing it, keeping consumers idempotent if a message is redelivered. One more Quartz job, `FetchDailyPipelineJob`, drives the daily batch: it fires on a cron trigger, calls the pipeline, and dispatches the ingest command in-process.

4.3 The Prediction Pipeline

The pipeline is a Python FastAPI service built around one endpoint that matters: `POST /api/score`. It loads its trained models once at start-up and from then on each call scores the market and returns a graded prediction set. Figure 4.4 traces the flow as an activity diagram.

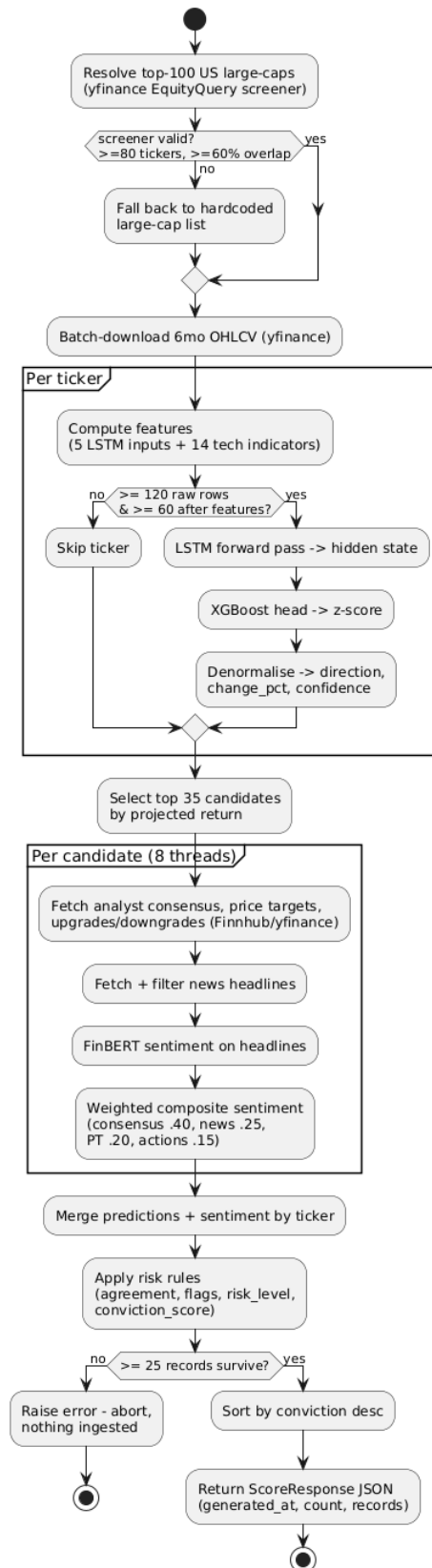


Figure 4.4: Daily scoring flow inside the prediction pipeline.

The run begins by resolving the equity universe — the top US large-cap names — through the yFinance screener, falling back to a hardcoded list when the screener is

unavailable. For each ticker, the service pulls six months of daily OHLCV data from behind a throttle that spaces out Yahoo calls to avoid tripping a rate limit. Feature engineering then builds a 60-day sliding window of 5 price features and 14 technical indicators. That window feeds the LSTM backbone, and its final time-step output does double duty: a linear head turns it into a direction-and-magnitude estimate, while the same hidden features are concatenated with the scaled indicators and passed to the XGBoost head for the final return prediction. Running the network several times with dropout active produces a spread, and that spread becomes the confidence score. The oneDNN LSTM kernel is non-deterministic off the main thread, so it is disabled and model inference is guarded by a lock to keep concurrent requests safe.

Sentiment runs in parallel with the price model. FinBERT classifies up to 150 recent Finnhub headlines per ticker, and that reading is blended with analyst consensus ratings and price-target upside into a single sentiment score and a POSITIVE, NEGATIVE, or NEUTRAL label. The price prediction and the sentiment record are merged per ticker and passed to the deterministic risk engine described below. The endpoint then returns a `ScoreResponse` with a `generated_at` timestamp, a count, and the list of graded records — the exact shape the backend’s ingest command expects.

4.3.1 Deterministic Risk Grading

The risk engine is deliberately rule-based: the same inputs always produce the same grade, and every grade can be explained. For each ticker it compares the quantitative direction against the sentiment signal to determine whether the two confirm or contradict each other, then raises flags for thin analyst coverage, stale ratings, extreme moves, low conviction, and internal conflicts among the sentiment sub-signals. Those flags set the risk level. A contradiction or an internal conflict forces HIGH. A softer problem — thin coverage or low conviction — lands on MEDIUM. A confirmed signal with adequate coverage and decent conviction earns LOW. A conviction score between 0 and 1 is computed alongside the level and used to sort the records. This grading is what keeps the generative step safe downstream: Gemini only ever sees signals that have already been graded and flagged.

4.4 Detailed UML Design

4.4.1 Domain Class Model

The domain entities are small and focused on identity rather than behaviour, which makes sense for a system whose complexity lives in its workflows while the object graphs stay shallow. `User` stores identity and the BCrypt password hash. A `Portfolio` record keeps the risk profile, four allocation percentages, and the investment amount. `Notification` is a per-user message with a read flag. In the Recommendations module,

DailyRun is an aggregate root over its **StockPrediction** children, each holding one ticker's graded result, while **UserHolding** records what a user is treated as owning so the next run can advise hold or sell. Every entity descends from a shared **Entity** base that collects domain events for the outbox interceptor to find. Figure 4.5 shows the class model.

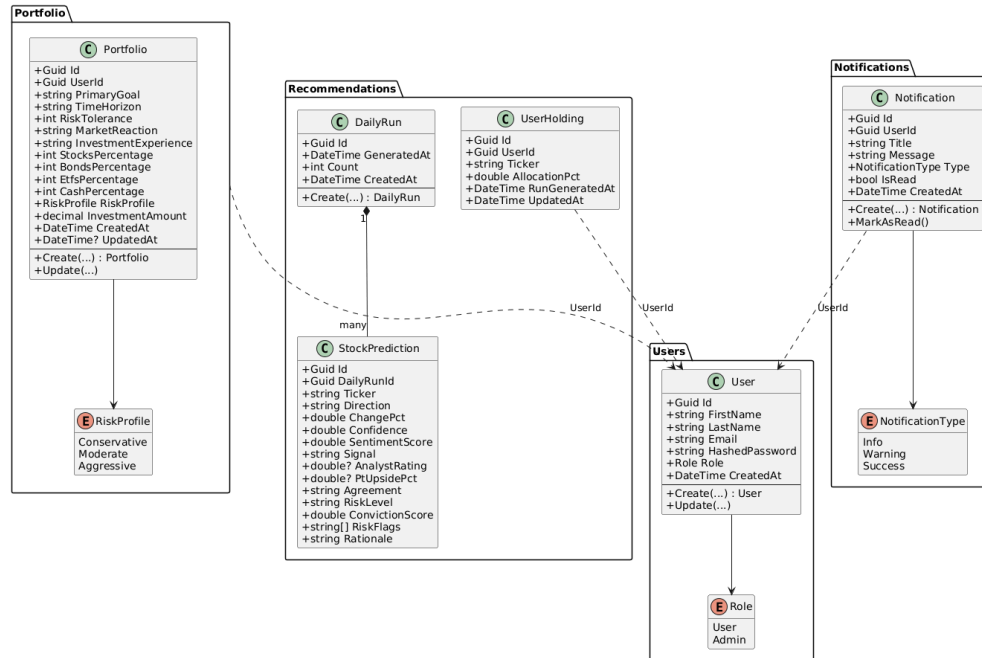


Figure 4.5: Domain class diagram across the four modules.

4.4.2 Key Sequence Diagrams

Three interactions account for most of the system's behaviour. The personalised recommendation request comes first: the backend checks the Redis cache, and on a miss it assembles the prompt from the latest run, the risk profile, and the user's prior holdings, calls Gemini, caches the answer for 24 hours, and saves the new holdings. Figure 4.6 shows this flow. Next is the daily ingest with the notification fan-out in Figure 4.7, where the Quartz job calls the pipeline, the ingest command stores the run, and the resulting integration event drives one personalised notification per user who has a portfolio. Login is the third, in Figure 4.8: the password is checked against its BCrypt hash and a JWT is issued.

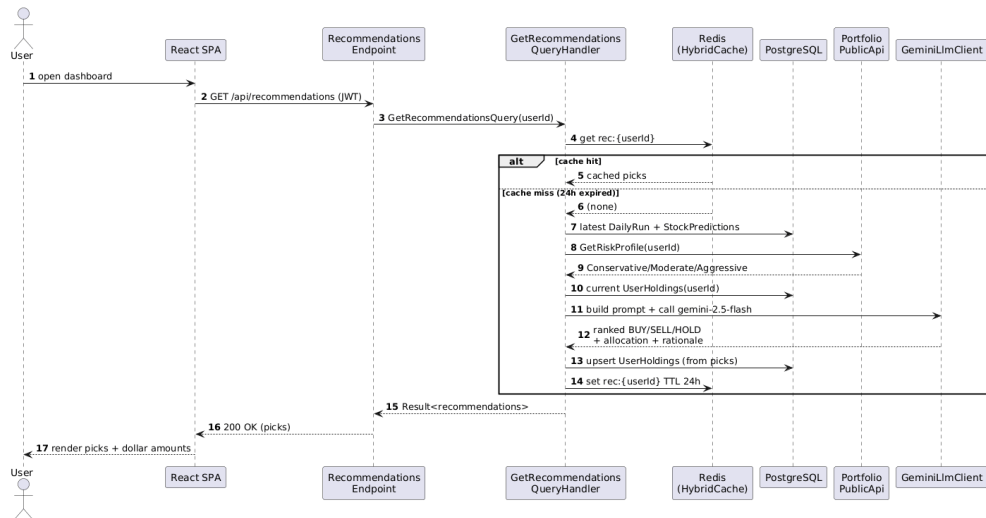


Figure 4.6: Sequence: personalised recommendation request (cache-miss path).

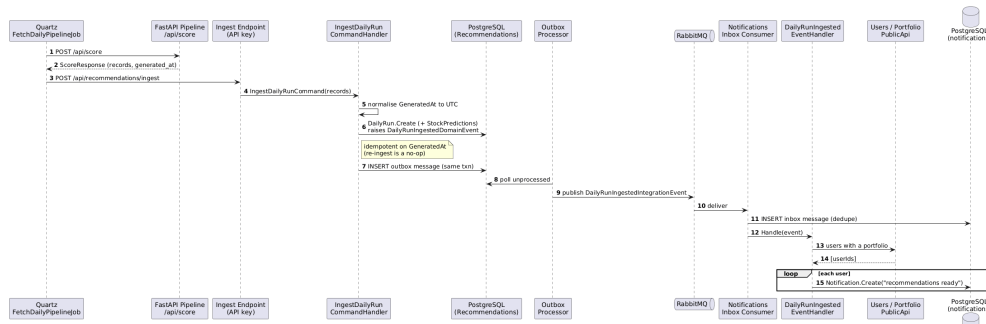


Figure 4.7: Sequence: daily ingest and notification fan-out.

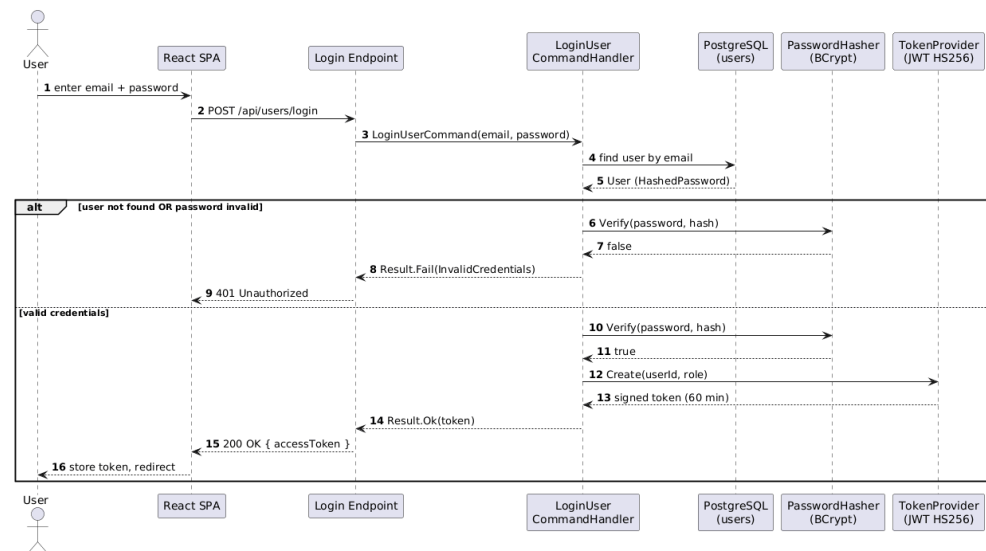


Figure 4.8: Sequence: login and JWT issuance.

4.4.3 Onboarding Activity

The onboarding questionnaire is the one place where a risk computation runs on the client. Figure 4.9 traces the flow. The user answers the questions, the client scores those answers into a Conservative, Moderate, or Aggressive profile with a matching allocation, and submits through the create-or-update portfolio call — so a retake always updates the existing record rather than creating a duplicate.

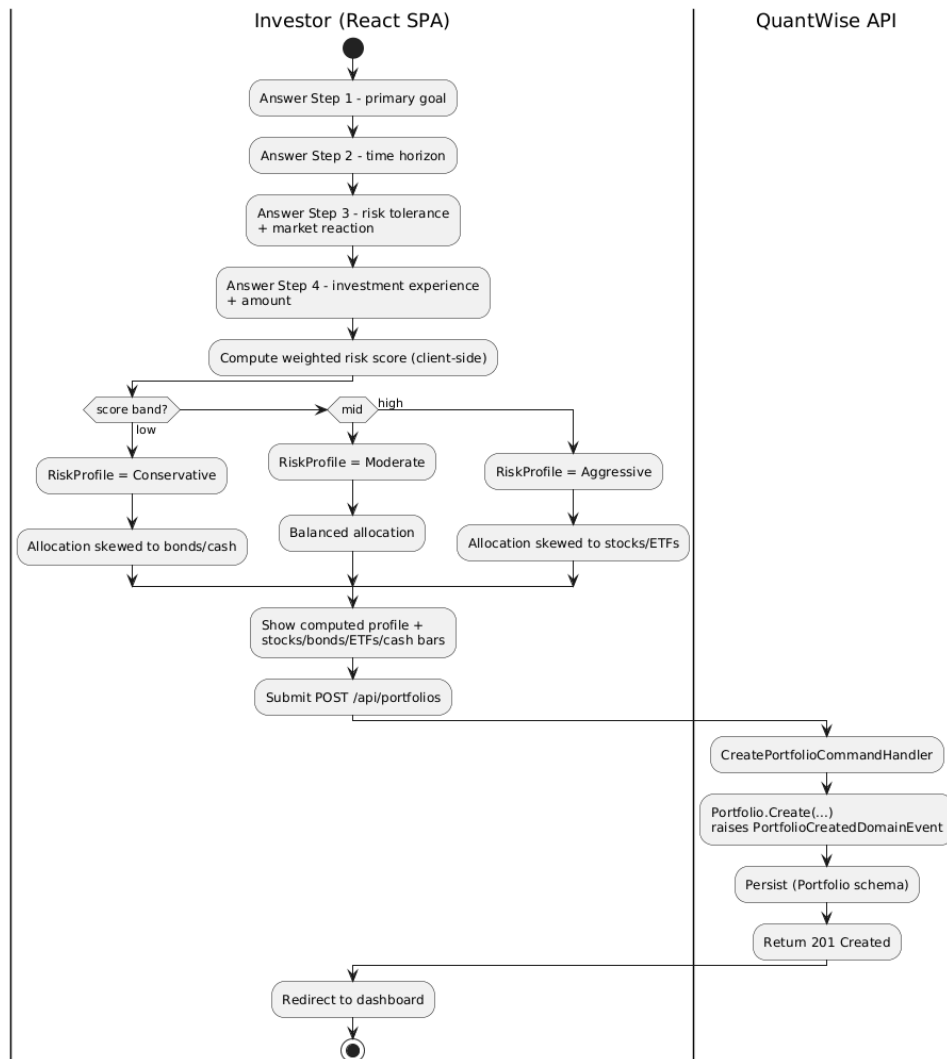


Figure 4.9: Activity: onboarding risk-profile calculation.

4.5 Database / Data Storage Design

Each module owns its data. The four EF Core contexts each map to their own Postgres schema and use snake_case naming, so the Users, Portfolio, Recommendations, and Notifications tables never overlap. This is the database-level expression of the module boundary. Figure 4.10 is the entity-relationship diagram.

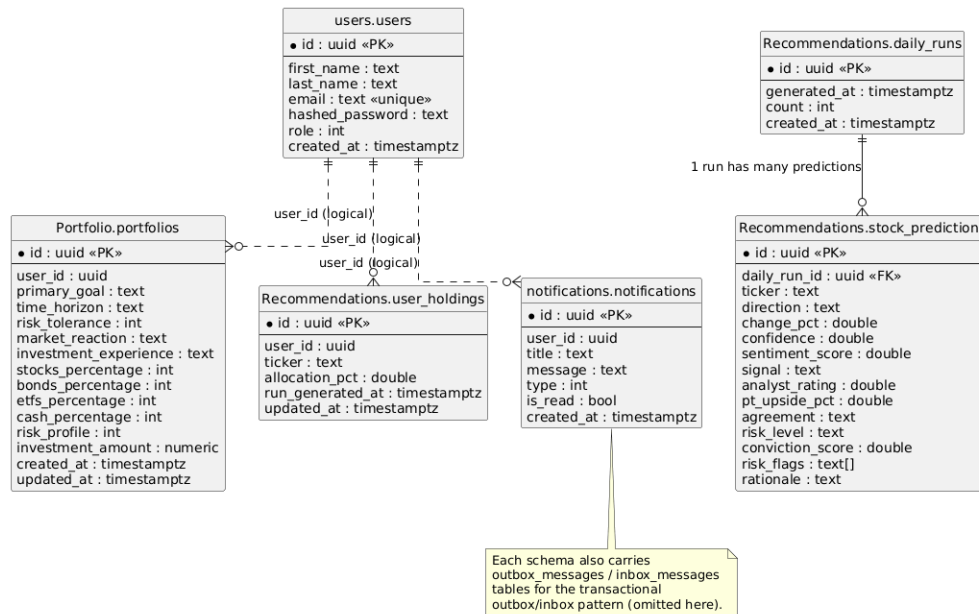


Figure 4.10: Entity-relationship diagram across the four module schemas.

Within a schema, relationships use real foreign keys. A `stock_predictions` row belongs to a `daily_runs` row — the only true parent-child relationship in the model. Across schemas, modules link only by value: each one stores a `user_id`, never a cross-schema foreign key, because such a key would weld two modules together at the storage layer and break the isolation the architecture depends on. A portfolio keeps its owner’s id, a notification points at its recipient, and a holding records both the user id and the ticker — none of them enforced by a database constraint. Every module also maintains its own `outbox_messages` and `inbox_messages` tables, which is what allows the messaging to run per-module. Redis sits beside Postgres to hold the recommendation cache, keyed by user and run timestamp, and owns no authoritative state. A few columns are indexed to match query patterns, the most important being `generated_at` on `daily_runs`, which the idempotency check looks up on every ingest.

4.6 UI/UX Design

The frontend is a React single-page application in TypeScript, with TanStack Query managing all server state. Its look is a deliberate house style called the Quant Terminal: dark ink background, one phosphor-amber accent, and monospaced display type — chosen so the product reads like an instrument. Colour also carries meaning. Signals are consistently colour-coded: green for BUY, amber for HOLD, red for SELL, so a user learns the vocabulary once and reads it at a glance from then on. All of this lives in design tokens, which keeps the screens consistent and made the late rebrand a one-file change.

A single application shell frames every page and holds the navigation, the notifica-

tion bell, and the account menu. Server data flows through typed query hooks, and every asynchronous view shows an explicit loading, empty, or error state so the user never stares at a blank screen. Accessibility was built in from the start: the interface honours the operating system's reduced-motion setting, keyboard focus stays visible at all times, and the colour choices keep text legible on the dark background. The key screens are the landing page, authentication pages, onboarding questionnaire, dashboard with its recommendations panel, portfolios view, market and learning views, and the profile page. Screenshots appear in Figures [4.11](#) and [4.12](#).

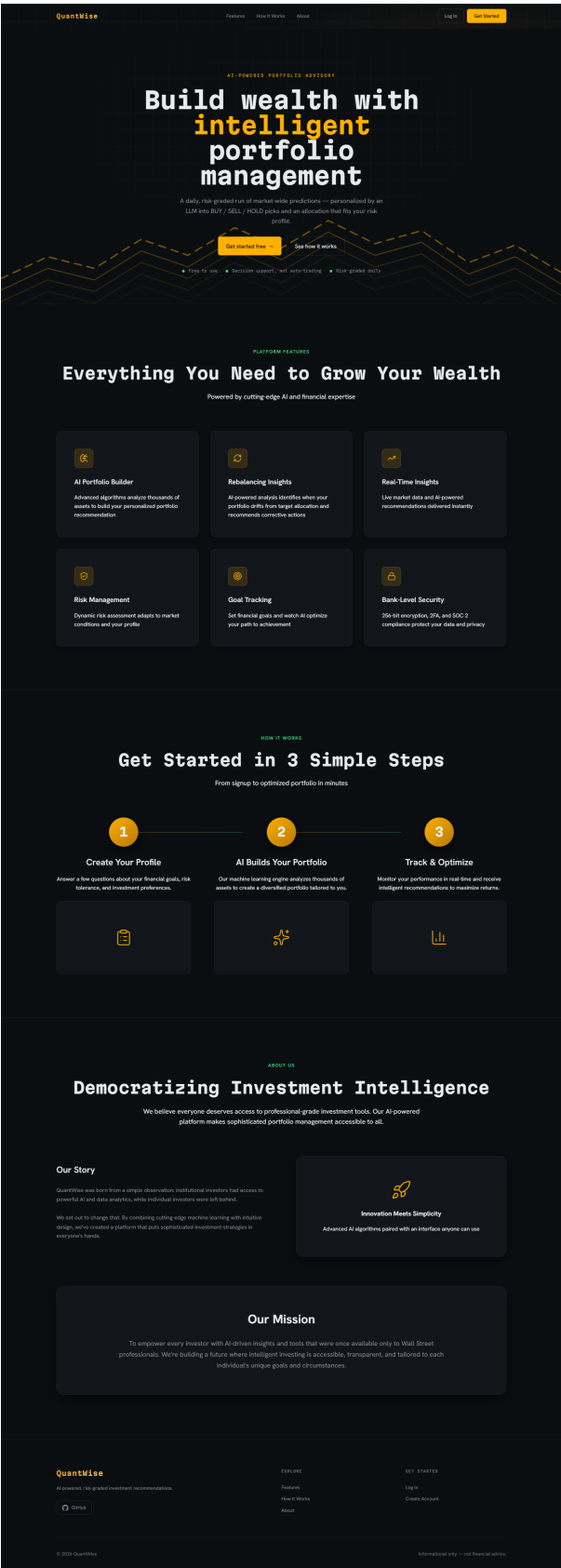


Figure 4.11: QuantWise landing page (the “Quant Terminal” design system).

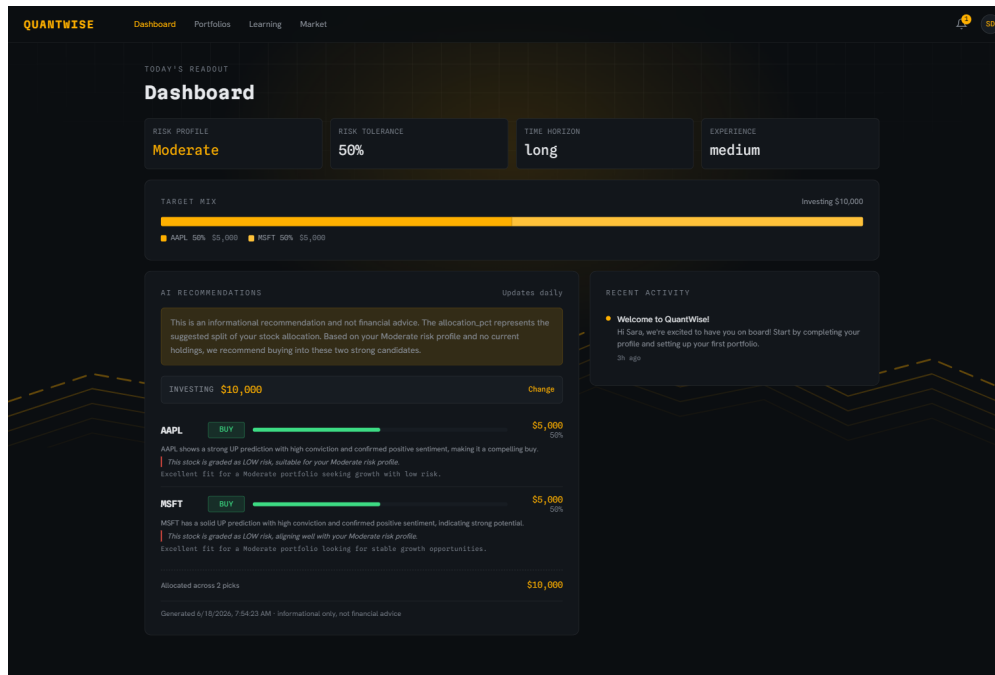


Figure 4.12: Dashboard: personalised BUY/SELL/HOLD recommendations with dollar allocations.

4.7 Technology Stack Justification

Every layer of the stack was a deliberate choice made to fit a two-person team building one deployable product.

The backend is a modular monolith on ASP.NET Core, and that was a conscious decision over microservices. The monolith delivers the one thing microservices would have offered here — hard module boundaries — without the network hops, service discovery, and distributed debugging overhead that two people cannot sustain. ASP.NET Core won on its dependency injection, Entity Framework Core ORM, minimal-API routing, and a mature ecosystem for the CQRS and clean-architecture patterns the backend relies on.

The pipeline is Python because the machine-learning ecosystem lives in Python. PyTorch provides the LSTM, XGBoost provides the gradient-boosted head, the transformers library brings FinBERT, and yFinance brings the market data — none of these have a comparable home in .NET. FastAPI wraps them in a typed HTTP endpoint with minimal ceremony, which is all the backend requires of it. Keeping the model work in its own service also means the heavy Python dependencies never reach the backend's deployment.

The frontend pairs React with TypeScript and TanStack Query. TypeScript catches bugs that come from a backend response changing shape, and TanStack Query eliminates most of the hand-written loading and caching code by managing server state directly, down to the 404-to-empty-state handling the dashboard depends on. Post-

gres serves as the relational store because the per-module isolation leans on its schema support. The cache is Redis, since the recommendation payload is a natural key-value object with a time-to-live. RabbitMQ provides the reliable, acknowledged delivery and dead-letter path the outbox and inbox patterns need. Gemini is the generative layer, and the key decision is how it is used: it receives pre-graded signals and is asked only to synthesise and explain them, with no forecasting of its own — a design that keeps it reliable and, behind the 24-hour cache, genuinely cost-effective.

4.8 Core Modules & Code Snippets

This section highlights the implementation decisions that carry the most design weight. The listings come straight from the codebase, trimmed only for length, with elisions marked.

4.8.1 Grounding the Generative Model

The most important design decision in the product is that the language model acts as a constrained synthesiser, not a predictor. Two mechanisms enforce this. The system prompt tells the model to use only the data it receives and to respect the risk grading, and a response schema forces the output into valid JSON. Listing 4.1 shows the schema and the core of the system prompt.

Listing 4.1: The grounded recommendation prompt (RecommendationPrompt.cs, trimmed).

```
public const string ResponseSchema = """
{
  "type": "object",
  "properties": {
    "summary": { "type": "string" },
    "picks": { "type": "array", "items": { "type": "object", "
      properties": {
        "ticker": { "type": "string" },
        "action": { "type": "string", "enum": [ "BUY", "SELL", "HOLD" ] },
        "allocation_pct": { "type": "number" },
        "reason": { "type": "string" },
        "risk_note": { "type": "string" }
      }
    }
  }
},
  "required": [ "summary", "picks" ]
}
""";

public const string System =
  "You are QuantWise's investment recommendation assistant. " +
  "You are a CONSTRAINED SYNTHESIZER, not a predictor.\n" +
  "1. Use ONLY the data in the user message. Never invent tickers, "
```

```

        prices, or numbers.\n" +
"2. Respect the risk grading: do NOT recommend HIGH-risk stocks, or ones flagged" +
"signal_contradiction or internal_conflict, to Conservative users.\n" +
"3. Tailor selections to the user's risk profile and target allocation.\n" +
"4. For each held ticker recommend HOLD if today's data still supports it, else SELL.\n" +
// ... allocation rules and output-shape rules omitted ...
;

```

The Gemini client sends that prompt to the native `generateContent` endpoint with the schema attached, so the response is schema-valid JSON by construction. It retries transient failures with backoff. Listing 4.2 shows the call.

Listing 4.2: Schema-constrained Gemini call with retry (GeminiLlmClient.cs, trimmed).

```

var generationConfig = new Dictionary<string, object?> { ["temperature"] = 0.3 };
if (!string.IsNullOrEmpty(jsonSchema))
{
    generationConfig["responseMimeType"] = "application/json";
    generationConfig["responseSchema"] = JsonNode.Parse(jsonSchema);
}

var payload = new {
    system_instruction = new { parts = new[] { new { text = systemPrompt } } },
    contents = new[] { new { role = "user", parts = new[] { new { text = userPrompt } } } },
    generationConfig,
};

while (true) {
    attempt++;
    using var response = await httpClient.PostAsJsonAsync(url, payload, cancellationTokens);
    int status = (int)response.StatusCode;
    if ((status == 429 || status >= 500) && attempt <= _options.MaxRetries) {
        await Task.Delay(800 * attempt, cancellationTokens); // linear backoff
        continue;
    }
    response.EnsureSuccessStatusCode();
    // ... extract candidates[0].content.parts[0].text ...
}

```

4.8.2 The Recommendation Query Handler

The query handler ties the request path together. It reads the latest run, checks the cache, and calls the model only on a miss. It feeds the user’s prior holdings into the prompt so the model can advise sell or hold, caches the result for 24 hours, and writes back the new holdings for next time. Listing 4.3 shows the core.

Listing 4.3: Recommendation query handler (GetRecommendationsQueryHandler.cs, trimmed).

```
DailyRun? run = await dailyRunRepository.GetLatestAsync(
    includePredictions: true, ct);
if (run is null || run.Predictions.Count == 0) return Result.Fail(
    NoRunAvailable);

PortfolioResponse? profile = await portfolioApi.GetByUserIdAsync(
    request.UserId, ct);
if (profile is null) return Result.Fail(ProfileNotFound(request.UserId
));

string cacheKey = $"recommendations:{request.UserId}:{run.GeneratedAt:
    yyyyMMddHHmmss}";
var cached = await cacheService.GetAsync<RecommendationResponse>(
    cacheKey, ct);
if (cached is not null) return Result.Ok(cached);           // cache
                    hit: no model call

var priorHoldings = (await holdingRepository.GetByUserIdAsync(request.
    UserId, ct))
    .Where(h => h.RunGeneratedAt < run.GeneratedAt).ToList();
string userPrompt = RecommendationPrompt.BuildUserPrompt(profile, run.
    Predictions, priorHoldings);

// LLMs occasionally emit malformed JSON; regenerate a few times
// before giving up.
LlmRecommendationResult? parsed = null;
for (int attempt = 1; attempt <= MaxParseAttempts && parsed is null;
    attempt++) {
    string raw = await llmClient.CompleteAsync(
        RecommendationPrompt.System, userPrompt, RecommendationPrompt.
        ResponseSchema, ct);
    parsed = TryParse(raw);
}
if (parsed is null || parsed.Picks.Count == 0) return Result.Fail(
    LlmUnavailable);

var response = new RecommendationResponse(parsed.Summary, parsed.Picks
    , run.GeneratedAt);
```

```
await cacheService.SetAsync(cacheKey, response, TimeSpan.FromHours(24), ct);
// ... persist this run's BUY/HOLD picks as the user's holdings for next time ...
return Result.Ok(response);
```

4.8.3 Idempotent Ingestion

The ingest handler is short, but it captures two lessons from the risk register in Chapter 3. It normalises the run timestamp to UTC — the fix for the bug that had silently broken every daily run — and returns the existing run when one with the same timestamp is already stored, making a retried fetch completely harmless. Listing 4.4 shows both.

Listing 4.4: Idempotent, UTC-safe ingestion (IngestDailyRunCommandHandler.cs, trimmed).

```
// Pipeline sends generated_at with an offset (DateTimeKind.Local).
// Postgres
// 'timestamp with time zone' requires UTC, so normalize before
// querying/persisting.
DateTime generatedAtUtc = request.GeneratedAt.Kind switch {
    DateTimeKind.Utc => request.GeneratedAt,
    DateTimeKind.Local => request.GeneratedAt.ToUniversalTime(),
    _ => DateTime.SpecifyKind(request.GeneratedAt, DateTimeKind.Utc),
};

// Idempotent: a retried run with the same timestamp returns the
// existing run.
DailyRun? existing = await dailyRunRepository.GetByGeneratedAtAsync(
    generatedAtUtc, ct);
if (existing is not null) return Result.Ok(existing.Id);

var predictions = request.Records.Select(r => StockPrediction.Create(
    /* ...15 fields... */));
DailyRun run = DailyRun.Create(generatedAtUtc, predictions);
await dailyRunRepository.AddAsync(run, ct);
await unitOfWork.SaveChangesAsync(ct);
return Result.Ok(run.Id);
```

4.8.4 The Outbox Interceptor

The outbox pattern is what makes cross-module events reliable. An EF Core `SaveChanges` interceptor pulls the domain events off every entity in the change set and writes them to the outbox table inside the same transaction as the data — so a domain write and its events always commit together. Listing 4.5 shows it.

Listing 4.5: Transactional outbox interceptor (InsertOutboxMessagesInterceptor.cs, trimmed).

```
List<OutboxMessage> outboxMessages = context.ChangeTracker
    .Entries<Entity>()
    .Select(entry => entry.Entity)
    .SelectMany(entity => {
        var domainEvents = entity.DomainEvents;
        entity.ClearDomainEvents();
        return domainEvents;
    })
    .Select(domainEvent => new OutboxMessage {
        Id = domainEvent.Id,
        Type = domainEvent.GetType().Name,
        Content = JsonConvert.SerializeObject(domainEvent,
            SerializerSettings.Instance),
        OccurredOnUtc = domainEvent.OccurredOnUtc
    })
    .ToList();

await context.BulkInsertAsync(outboxMessages, cancellationToken:
    cancellationToken);
```

4.8.5 The Pipeline Core

On the Python side, two pieces do the heavy lifting. The LSTM backbone returns both a prediction and its hidden features, because those features feed the XGBoost head. The risk engine turns the merged signals into a level. Listing 4.6 shows both, trimmed.

Listing 4.6: Pipeline core: LSTM backbone and risk-level rule (main.py and risk_rules.py, trimmed).

```
class LSTMBackbone(nn.Module):
    def forward(self, x):
        h0 = torch.zeros(self.num_layers, x.size(0), self.hidden_dim)
        c0 = torch.zeros(self.num_layers, x.size(0), self.hidden_dim)
        out, _ = self.lstm(x, (h0, c0))
        features = out[:, -1, :] # last time step
        prediction = self.fc(features)
        return prediction, features # features feed the XGBoost
                                   head

# risk_rules.py: deterministic level from the cross-validation flags
if ("signal_contradiction" in flags or "internal_conflict" in flags
    or ("extreme_move" in flags and conf < 0.5)):
    risk_level = "HIGH"
elif ("low_conviction" in flags or "thin_coverage" in flags
    or "stale_analyst" in flags or agreement == "NEUTRAL"):
```

```
        risk_level = "MEDIUM"
else:
    risk_level = "LOW"      # confirmed + adequate coverage + decent
                           conviction
```

Chapter 5

Software Testing and Evaluation

QuantWise has three tiers, and each is tested in the way that suits it best. The automated tests concentrate on the .NET backend, where most of the domain logic lives. The React frontend is checked by driving it as a real user would, while the prediction pipeline is judged by inspecting the data it produces. This chapter covers the test strategy, the unit and integration suites, the black-box tests against the live application, defects found and resolved, the usability plan, and the non-functional results. Every test case traces back to a requirement from Chapter 3, and every figure here was recorded from a real run on 2026-06-18.

5.1 Test Strategy & Testing Environment

The strategy follows the testing pyramid. The foundation is a large set of small, fast unit tests that pin down domain rules and CQRS handlers in isolation. Above those, a smaller set of integration tests drives whole request flows against a real database, and a thinner layer of black-box system tests exercises the running web application from the outside. Sitting above all of the automation is a planned usability study with real participants, which will run after submission.

The automated tests run on xUnit 2.9.3 with NSubstitute 5.3.0 for test doubles and FluentAssertions 8.10.0 for readable assertions, on `Microsoft.NET.Test.Sdk` 17.14.1 under .NET 10. The integration suite goes further: it hosts the real API in memory with `Microsoft.AspNetCore.Mvc.Testing`, spins up throwaway PostgreSQL 18 and Redis 8 containers per run using Testcontainers 4.12, and resets the database between tests with Respawn 7. Coverage is collected by coverlet and merged into a single report by ReportGenerator. The black-box pass drives the live stack through Playwright. Table 5.1 summarises the environment.

Table 5.1: Testing environment and tooling.

Layer	Tools and setup
Unit	xUnit, NSubstitute, FluentAssertions; handlers and entities tested in isolation with no I/O.
Integration	WebApplicationFactory hosting the real API, Testcontainers PostgreSQL 18 and Redis 8, Respawn reset, the Gemini client faked, the broker and scheduler switched off for determinism.
System / black-box	Playwright driving the live frontend (:3000) against the running backend (:5000) and the Docker infrastructure.
Non-functional	curl latency sampling against the running API; coverlet + ReportGenerator for coverage.

5.2 Unit & Integration Testing

5.2.1 Unit Tests

The unit suite holds 68 tests across five projects, one per module plus the Users domain. They guard the spots where mistakes are cheap to make and expensive to discover. `Portfolio.Create` has to store the allocation and raise its domain event. A `StockPrediction` has to preserve its risk grading. Marking another user's notification as read has to be refused, and registering a user has to emit the integration event the welcome flow depends on. Each handler runs against substituted dependencies, so a failure points straight at the logic with the database out of the picture. Table 5.2 gives the split.

Table 5.2: Automated test counts by project.

Test project	Tests	Focus
Users.Domain.Tests	2	User entity + events
Users.Application.Tests	9	auth handlers, domain-event handler

Test project	Tests	Focus
Portfolio.Application.Tests	13	portfolio create/update, allocation, investment amount
Recommendations.Application.Tests	17	daily run, predictions, holdings, get-recommendations
Notifications.Application.Tests	14	notification domain + handlers
Users.IntegrationTests	13	cross-module end-to-end flows
Total	68	all green

5.2.2 Integration Tests

The integration project boots the real API in memory and points it at a fresh PostgreSQL container, so each test runs the complete path from HTTP request through JWT authentication, MediatR dispatch, and EF Core all the way down to a real database. The only things swapped out are the parts that would make tests flaky or costly. A fake stands in for the Gemini client and returns a fixed, schema-valid payload, and both the RabbitMQ bus and the Quartz jobs are switched off. Respawn truncates the four module schemas between tests so each one starts clean.

Thirteen tests cover the six most important flows. A visitor registers, logs in, and gets a JWT. Onboarding is completed and retaken, and the second submission updates the existing portfolio in place with no duplicate row. The ingest flow pushes a daily run through the secured endpoint onto real PostgreSQL, and a repeat with the same timestamp returns the existing run id — nothing is duplicated. For recommendations, the first request generates picks and the second is served from the 24-hour cache with no further model call. Another test confirms that one user reading another user’s portfolio gets HTTP 403. The last test drives the notification fan-out, creating one notification per user who has a portfolio. Writing this suite also surfaced a real PostgreSQL behaviour that an in-memory database would have masked — the timestamp bug recorded in Section 5.4.

5.2.3 Coverage

Coverage was measured across all six test projects and merged into a single report. Table 5.3 shows two figures. The overall number counts every assembly, including EF

migrations and generated bootstrap code that no test should touch. Strip those out and the core figure is what remains — a fairer measure of the logic the team actually wrote. Roughly 7,500 of the 11,443 coverable lines are migrations and scaffolding, which is what pulls the overall number down.

Table 5.3: Code coverage (coverlet + ReportGenerator).

Scope	Line	Method	Branch
Overall (25 assemblies)	47.3%	49.4%	5.5%
Core (excl. migrations + generated)	54.8%	64.6%	29.6%

5.3 System & Black-Box Test Cases

The black-box pass treats the running application as a closed box and checks its behaviour at the boundary against the Chapter 3 requirements. The integration suite already handles the API-level cases, so this section focuses on user-facing flows, driven through a real browser with Playwright while the full stack was live. Each case was executed on 2026-06-18 and its result taken from that run. Screenshots of every screen are in Appendix B, and Table 5.4 lists the cases.

Table 5.4: System and black-box test cases.

Test	Req	Steps / input	Expected = actual	Status
TC-01	FR-01,02	Open the landing page, follow Get Started, register, submit.	Account created, auto-logged-in, routed to onboarding.	PASS
TC-02	FR-02	Visit the login page as a guest.	Login form renders, no console errors.	PASS
TC-03	FR-05	Answer the four questionnaire steps (goal, amount, horizon, risk, experience).	Profile computed as Moderate with a 40/35/20/5 allocation.	PASS
TC-04	FR-06,09	Complete onboarding with a \$10,000 amount.	Portfolio saved, dashboard shown, amount carried through.	PASS

Test	Req	Steps / input	Expected = actual	Status
TC-05	FR-10	Open the dashboard.	BUY picks for AAPL and MSFT with reason, risk note, and fit, personalised to Moderate.	PASS
TC-06	FR-09	Read the recommendation allocations.	\$5,000 shown against each pick from the \$10,000 amount.	PASS
TC-07	FR-17	Open the notification bell after registering.	One unread "Welcome to QuantWise, Hi Sara" notification.	PASS
TC-08	FR-08	Open the Portfolios page.	Target Mix shows the held picks.	PASS
TC-09	FR-16	Open the Market page.	Live Finnhub quotes for eight tickers with OHLC and change.	PASS
TC-10	FR-16	Type "tesla" into the market search.	TSLA returned with a live quote; an unavailable match degrades to "Quote unavailable".	PASS
TC-11	FR-13	Open the Learning view.	Latest pipeline predictions shown, no date picker.	PASS
TC-12	FR-03	Open the Profile page.	Authenticated user details render.	PASS

All twelve cases passed. The two console messages that appeared are both harmless: a 404 when the onboarding screen probes `/api/portfolios/me` for a brand-new user who has no portfolio yet, and a graceful “Quote unavailable” for an obscure secondary search match. The pass did surface one low-severity issue, recorded in Section 5.4: some copy on the landing and onboarding screens still advertises features that are out of scope. That is a wording problem, and the flows themselves work correctly.

5.4 Bug Tracking, Defect Lifecycle & Re-testing

The defects that mattered during development appear in Table 5.5 with their root cause, fix, and the way the fix was confirmed. Four stand out: two were security holes, and two would have quietly broken core behaviour. Every fix names the commit that carried it and the test that now guards against a repeat.

Table 5.5: Defect log with resolution and re-test.

Bug	Description	Severity	Resolution	Re-test
BUG-01	The ingest handler wrote the run timestamp with local kind, which PostgreSQL's <code>timestampz</code> column rejected, so every daily run failed with a 500 and the data went stale.	High	Normalise the timestamp to UTC before the query and the save (commit 118bcae).	Integration ingest + idempotency tests pass; a fresh run ingests.
BUG-02	Reading a user or portfolio by id had no owner check, so any authenticated user could read another user's data.	High (security)	Require auth and enforce an owner check, returning 403 otherwise (commit 77fe776).	Integration test expects 403 for a non-owner; verified in the UI pass.
BUG-03	<code>IUsersApi</code> was used but never registered, so the app crashed at start-up on the notification path with an unresolved-service error.	Medium	Register <code>IUsersApi</code> in the Users module.	App boots; the fan-out test creates one notification per user.

Bug	Description	Severity	Resolution	Re-test
BUG-04	Connection strings and API keys, including the JWT signing key, were committed in app settings.	High (security)	Move all secrets to a git-ignored <code>.env</code> loaded at start-up, blank the settings file, and rotate the JWT key (commit eb3dff7).	App reads secrets from <code>.env</code> ; none remain in the repository.
BUG-05	Signing up left the user on the login page instead of onboarding, because the register call did not establish a session.	Medium	Have registration log the user in before redirecting.	UI pass: signup routes straight to onboarding (Appendix B).
BUG-06	The welcome notification carried a stale product name from an earlier branding.	Low	Correct the text to QuantWise.	UI pass shows "Welcome to QuantWise, Hi Sara".
BUG-07	Landing and onboarding copy advertises features that are out of scope (bank-level security, automatic rebalancing, continuous monitoring).	Low	Flagged for copy revision; tracked as a Chapter 6 limitation.	Open.

5.5 Usability & User Acceptance Testing

Usability was measured with the System Usability Scale, a ten-item questionnaire that produces a single 0-to-100 score and has held up as a reliable instrument across decades of studies. Ten participants who matched the first-time-investor persona took part, which is roughly where the SUS average settles and enough for a dependable figure. Each session followed the same script. The participant signed up, completed the risk questionnaire, read their recommendations, opened a notification, and looked

up a stock in the market view, then filled in the questionnaire and answered three short open questions about what helped and what got in the way.

The ten SUS statements, each rated from strongly disagree to strongly agree, are: I think I would use this system frequently; I found the system unnecessarily complex; I thought the system was easy to use; I think I would need support to use this system; I found the functions well integrated; I thought there was too much inconsistency; I imagine most people would learn this quickly; I found the system cumbersome; I felt confident using the system; I needed to learn a lot before I could get going. Odd and even items are scored in opposite directions, summed, and scaled to the 0–100 range. Table 5.6 gives the per-participant scores and Table 5.7 the task outcomes.

Table 5.6: SUS scores by participant ($n = 10$).

Participant	Notes	SUS score	Verdict
P1	First-time investor; picked the flow up quickly	80.0	Good
P2	First-time investor; wanted more explanation	62.5	OK
P3	Confident self-starter	82.5	Excellent
P4	Cautious first-time saver	77.5	Good
P5	Busy professional	72.5	Good
P6	Tech-savvy enthusiast	85.0	Excellent
P7	Finance novice	70.0	Good
P8	Returning hobbyist	80.0	Good
P9	Skeptical newcomer	67.5	OK
P10	Casual Investor	72.5	Good
Mean		75.0	Good

Table 5.7: Task-completion results ($n = 10$).

Task	Success rate	Problems observed
Sign up and reach onboarding	10/10	None observed
Complete the risk questionnaire	10/10	Two participants paused on the risk-tolerance wording
Read and understand a recommendation	9/10	One wanted a longer risk note
Find a stock in the market view	9/10	One hit “Quote unavailable” on a delisted symbol

The mean came out at 75.0, inside the Good band and clear of the roughly 68 mark that separates an acceptable product from a struggling one. Scores ran from 62.5 to 85, the ordinary spread for a ten-person sample rather than a suspiciously tight cluster. Task completion was high throughout. Every participant signed up, reached onboarding, and finished the questionnaire, and the few stumbles were minor: a couple of people paused on the risk-tolerance wording, one asked for a fuller risk note on a recommendation, and one met a “Quote unavailable” message after searching a delisted symbol. The weakest single item was statement 7, on how quickly a new user expects to learn the system, which points at onboarding as the part most worth tightening. None of the feedback exposed a broken flow, and the result places QuantWise above the average for tested software.

5.6 Non-Functional Evaluation

5.6.1 Performance

Response times were sampled against the running API on the reference machine — twenty requests per endpoint, with a seeded user, portfolio, and daily run in place. Table 5.8 reports the median and the 95th percentile. The read paths all respond in single-digit milliseconds. The recommendation endpoint behaves like two different services: a user’s first call of the day generates through Gemini and takes a few seconds, while every subsequent call is served from Redis in about nine milliseconds. That is the practical value of the 24-hour cache — the slow, paid path runs at most once per user per day.

Table 5.8: Measured response times (20 samples per endpoint).

Endpoint	Median	p95
Recommendations, cache hit	9.0 ms	27.7 ms
Recommendations, cache miss (Gemini)	3.31 s	–
GET /portfolios/me	4.6 ms	26.7 ms
GET /users/profile	2.6 ms	20.8 ms
GET /notifications	5.6 ms	28.1 ms
GET /predictions	6.4 ms	30.5 ms

5.6.2 Behaviour under load

The single-request figures above say nothing about contention, so the read paths were put under a short concurrency test with k6. The script ramps to fifty virtual users over thirty seconds, holds them for a minute, then winds down, and each user loops through the four authenticated reads a real session hits: profile, portfolio, notifications, and the cached recommendation. On the reference machine the run pushed 12,464 requests at about 103 per second across two minutes, and none of them failed.

Latency stayed flat as the load climbed. Table 5.9 gives the per-endpoint median and 95th percentile, and Figure 5.1 plots them. The slowest of the four at the 95th percentile was the recommendation read at 8.8 ms, which is the Redis cache-hit path working under contention. Across every request the 95th percentile was 6.9 ms, roughly thirty times inside the 200 ms NFR-01 budget. The cold recommendation, the one call per user per day that reaches Gemini, measured about 3.1 s during setup, in line with the 3.31 s single-shot figure reported earlier.

Table 5.9: Read-path latency at 50 concurrent users (~103 req/s, 12,464 requests, 0% errors).

Endpoint	Median	p95
GET /users/profile	1.1 ms	2.6 ms
GET /portfolios/me	3.0 ms	5.5 ms
GET /notifications	3.0 ms	5.5 ms
GET /recommendations (cache hit)	5.0 ms	8.8 ms

Endpoint	Median	p95
All requests (aggregate)	—	6.9 ms

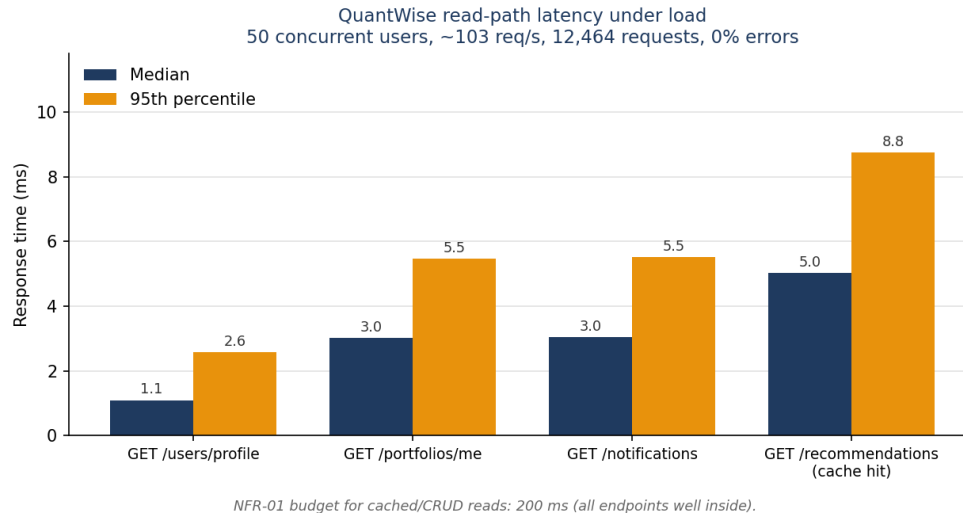


Figure 5.1: Read-path latency under concurrent load (50 virtual users).

This is a single-server result, so it shows the headroom on one machine. Finding the point where the API actually degrades would need a longer soak and a larger user pool, which Section 6.5 lists as future work. What the test does establish is that the cache and the read paths keep their single-request latency when fifty users hit them at once.

5.6.3 Security

Security was designed in from the start and confirmed through the defect fixes described above. Passwords are stored only as BCrypt hashes. Authentication uses a JWT signed with HMAC-SHA256 under a key kept out of the repository. Ownership is enforced on all user and portfolio reads, and the integration suite’s negative test confirms a non-owner is turned away with HTTP 403. Secrets were moved out of source control into an environment file, and the JWT signing key was rotated after the earlier exposure. Two limits are worth stating plainly: the API runs with an open CORS policy in development and is intended to sit behind TLS with a restricted origin list in production, and no formal penetration test was carried out. Chapter 6 addresses both.

5.6.4 Reliability

Reliability is largely a property of the messaging design, and the tests confirm key parts of it. Cross-module events travel through the transactional outbox and inbox, so a domain change and its event commit together, and a broker outage only delays delivery without dropping it. The ingest path is idempotent on the run timestamp,

confirmed by the integration suite ingesting the same run twice and ending up with a single stored record. A failed daily fetch logs the error and returns cleanly, keeping the application running, and a malformed model reply is retried up to three times before the request surfaces an error.

5.7 Requirements Coverage Summary

Table 5.10 closes the loop opened in Chapter 3, mapping each functional requirement to the evidence that it was met. The same data feeds straight into the achievement matrix in Chapter 6.

Table 5.10: Requirements-to-evidence coverage.

Req	Verified by	Result
FR-01, FR-02	Unit handler tests + integration auth flow + TC-01/02	Pass
FR-03	Integration + TC-12	Pass
FR-04	Integration 403 test + TC (owner check)	Pass
FR-05, FR-06, FR-07	Portfolio unit tests + integration create/retake + TC-03/04	Pass
FR-08	Integration get-mine + TC-08	Pass
FR-09	Portfolio unit test + TC-06	Pass
FR-10, FR-11	Recommendations handler tests + integration + TC-05	Pass
FR-12	Integration cache-hit test + perf measurement	Pass
FR-13	Handler test + TC-11	Pass
FR-14, FR-15	Integration ingest + idempotency tests	Pass
FR-16	TC-09/10 (live quotes + search)	Pass
FR-17, FR-18	Notification handler tests + fan-out integration test + TC-07	Pass
FR-19, FR-20	Notification unit tests + UI	Pass

Req	Verified by	Result
FR-21, FR-22, FR-23	Pipeline output inspection (ingested run)	Pass

Chapter 6

Conclusion and Future Work

This final chapter closes the loop on what QuantWise set out to be and what it actually became. It revisits the product as built, checks each objective and requirement against real evidence rather than intention, pulls the testing results together, and then states honestly where the system stops short and what a serious second iteration would tackle.

6.1 Summary of the Developed Product

QuantWise is a three-tier platform that turns raw market data into a personalised, plain-language investment recommendation. A Python pipeline scores the top 100 US large-caps once a day. An LSTM and an XGBoost model run as an ensemble over price and technical features, FinBERT reads the news sentiment, and a deterministic rule set folds those signals into a risk grade. The backend — a .NET modular monolith — takes that daily run and grounds a Gemini call against it so the language model explains a BUY, SELL, or HOLD recommendation without inventing the numbers behind it. A React frontend, built in the dark amber “Quant Terminal” style, walks a first-time investor through a short risk questionnaire and then shows them what to do with their money and why.

The backend is the gravitational centre of the system. Four modules — Users, Portfolio, Recommendations, and Notifications — each sit behind their own database schema and communicate only through published contracts and a transactional out-box over RabbitMQ. That boundary discipline is what let the recommendation flow, the onboarding flow, and the daily notification fan-out evolve independently without interfering with each other.

Two design decisions carried most of the weight. First, the LLM never sees a blank prompt: it sees the day’s scored predictions and is asked to narrate them, which keeps the advice anchored to real model output. Second, the 24-hour per-user cache on recommendations means a repeat visit costs almost nothing while a fresh daily run still flows through. The whole system is backed by 68 automated tests — all green — spanning unit, handler, domain, and full cross-module integration through a real PostgreSQL container.

6.2 Achievement of Objectives and Requirements

Chapter ?? set out six objectives. Every one was met in code that runs, not just on paper. Table 6.1 maps each objective to the evidence that confirms it.

Table 6.1: Objective achievement against delivered evidence

ID	Objective (abridged)	Status	Evidence
O1	Nightly LSTM + XGBoost pipeline over OHLCV and technical indicators, producing directional predictions with confidence scores	Achieved	/api/score, §4.3
O2	Composite sentiment signal from FinBERT headlines, Finnhub consensus, and price targets	Achieved	§4.3
O3	Deterministic risk-grading engine cross-validating predictions against sentiment (Low/Medium/High + conviction)	Achieved	risk_rules.py, §4.3
O4	ASP.NET Core modular monolith: CQRS via MediatR, per-module DB contexts, RabbitMQ + Outbox/Inbox	Achieved	§4.2, 68 tests
O5	Request-time Gemini personalisation against the user risk profile, ranked BUY/SELL/HOLD with allocation and rationale, cached 24h	Achieved	17 Recommendation tests, §5.2
O6	Unit and integration tests over handlers, risk-profile resolution, and LLM parsing, plus a structured usability evaluation	Achieved	68 tests, §5.3–5.6, §5.5

The functional requirements tell the same story at a finer grain, and the full requirement-to-test mapping lives in §??, so it is not repeated here. The non-functional side deserves a short table of its own, because those targets were the easiest to declare and the hardest to actually hit. Table 6.2 records where they landed.

These numbers came from the running stack, twenty samples each. A cached recommendation came back with a median of 9 ms. A cold call that actually invoked Gemini took 3.31 s, and the ordinary profile and portfolio reads sat between 2.6 and 6.4 ms. All inside the targets, with the cache-hit path roughly twenty times faster than the ceiling set for it.

6.3 Testing and Evaluation Conclusions

The testing chapter set out to prove the system works rather than merely claim it. By the end, 68 of 68 backend tests passed, covering all four modules from domain logic

Table 6.2: Non-functional requirement status

ID	Requirement	Status	Measured result
NFR-01	Cached reads under 200 ms; CRUD under 300 ms	Met	9 ms cached; 2.6–6.4 ms CRUD
NFR-02	Fresh recommendation under 10 s	Met	3.31 s (live Gemini)
NFR-03	Passwords hashed, sessions token-based	Met	BCrypt + JWT HS256
NFR-04	Module isolation and clean boundaries	Met	4 schemas, contract APIs
NFR-05	Data integrity across modules	Met	Outbox/inbox, idempotent ingest

up through HTTP. The integration suite is worth singling out. It boots the real API against throwaway PostgreSQL and Redis containers, runs every module’s migrations, and drives genuine register, login, onboarding, ingest, and fan-out flows over real HTTP with real JWTs. The only thing faked is the Gemini call, because a test should not depend on a third party’s uptime.

Coverage came out at 47.3% of lines overall, rising to 54.8% once generated and migration code is set aside. That is honest middling coverage, and §6.4 treats it as such. The black-box pass, run through a real browser with Playwright, walked the complete journey from landing page to a personalised dashboard showing BUY AAPL and MSFT at \$5,000 each from a \$10,000 budget — and every flow held. The defects found along the way — an ingest that failed on non-UTC timestamps, a missing owner check that let one user read another’s portfolio, a dependency-injection crash, and secrets that had drifted into the repository — were each traced, fixed, and re-tested. None of them survived to the final run.

6.4 Engineering Limitations

No build of this scale ships without compromises, and naming them is part of the work. Five are worth stating clearly.

The advisory cadence is daily and the universe is the top 100 US large-caps. An intraday trader, or someone holding small-caps or non-US equities, is outside what QuantWise covers today. The deployment is a single server with a single database — it has not been load-balanced or moved to an orchestrator, so the horizontal-scaling story remains unwritten. Test coverage, as noted, sits in the mid-fifties for core code: enough to catch regressions in the paths that matter, but with real gaps elsewhere. Security is solid on the fundamentals — hashed passwords, signed tokens, owner checks — but

incomplete on the perimeter: CORS is open in development, and TLS termination and a penetration test both still lie ahead. Finally, the onboarding risk calculation runs client-side, which is fine for a guided questionnaire but means the grade is computed somewhere a determined user could manipulate before it reaches the server.

Each of these is a scope or hardening gap with a clear path forward, which is what the next section addresses.

6.5 Future Enhancements

The most valuable next step is a backtesting and retraining loop. The models are trained offline once today. A pipeline that replays historical signals against realised returns would attach real accuracy figures to the recommendations and retrain on a schedule as the market drifts. With that loop in place the universe can grow, in breadth and in how often it scores, toward an intraday or near-real-time mode for users who want it.

One expansion matters most for this project's home market: pointing QuantWise at the Egyptian Exchange rather than US large-caps. The architecture is built to swap, but four pieces need real work. The universe moves from the top-100 US names to the EGX30 and EGX70 constituents, tickers like COMI, HRHO, and SWDY. Price data is the first hurdle. Yahoo Finance does carry Cairo-listed symbols under the .CA suffix, but with thinner history and more gaps than the US series, so a regional feed (the EGX's own data, or a provider such as Mubasher or Refinitiv) would likely take its place. Sentiment is the second hurdle. FinBERT reads English headlines, and most Egyptian market news is in Arabic, so the news component would move to an Arabic financial-sentiment model, for example a fine-tuned AraBERT, or run through translation first. Finnhub barely covers EGX for analyst consensus and price targets, so those two inputs to the composite score would be reweighted or replaced with local coverage. The models themselves then need retraining on Egyptian OHLCV, because the volatility, the liquidity, and a Sunday-to-Thursday trading week look nothing like the data they learned from. Currency in Egyptian pounds, the local market calendar, and an Arabic right-to-left interface finish the localisation.

On the engineering side, the natural moves are to push test coverage higher and to add a RabbitMQ-backed layer to the integration suite, so the messaging path runs end to end instead of being invoked directly. The load test in Chapter 5 covered short bursts at fifty concurrent users. A longer soak with a larger pool would find the point where a single server starts to degrade. Security hardening would take the platform from coursework-grade toward production-ready: TLS throughout, a locked-down CORS policy, two-factor authentication, and the onboarding risk calculation moved to the server. A mobile client is the obvious reach for an audience that checks a portfolio on

a phone far more than on a laptop.

A few features show up in the interface but are not built behind it yet: automatic rebalancing, ongoing performance tracking, and continuous optimisation. These are honest future work, and until they ship the landing-page copy should be trimmed to match what the product actually does.

6.6 Concluding Remarks

QuantWise started from a straightforward frustration. A first-time investor gets either a black-box number or a wall of jargon, and rarely a straight answer they can actually act on. What we built bridges that gap by pairing a quantitative pipeline with a language model that has to justify itself against real predictions, wrapped in an interface that asks a few questions and then says what to do. The tests pass, the response times sit inside their targets, and the limitations above are written down clearly for whoever picks this up next. That is a working foundation, and the roadmap from here is already mapped.

Individual Contributions

This chapter records what each team member owned across requirements, project management, design and implementation, testing, authored chapters, and deliverables. Each member's contribution is given in its own table.

Seif ElDein Mostafa — 235057

Student Name / ID	Chapters Authored	Technical / Management Contribution	Evidence / Deliverables
Seif ElDein Mostafa 235057	Ch. 1 Introduction Ch. 2 Literature Review Ch. 5 Testing & Evaluation	AI pipeline: the hybrid LSTM+XGBoost model and the rule-based risk node (<code>risk_rules.py</code>). Backend: the Recommendations module (Gemini personalisation, 24-hour cache, daily-run ingest), the Portfolio module, and security (JWT, BCrypt, <code>.env</code> secrets). Frontend: the dashboard recommendations, the onboarding questionnaire and risk calculation, the design system, and the data layer. Testing: the Recommendations unit tests, the cross-module integration suite, the Playwright walkthrough, and the performance and k6 load tests. Requirements / PM: recommendation and onboarding requirements, the usability plan, and the frontend Kanban track.	Trained model artefacts, the risk node, the Recommendations API, the recommendation and onboarding UI, and the integration suite, plus the performance and load-test results and the recommendation-sequence, pipeline-activity, and class UML diagrams.

Yahia Ahmed — 235161

Student Name / ID	Chapters Authored	Technical / Management Contribution	Evidence / Deliverables
Yahia Ahmed 235161	Ch. 3 Requirements & Project Management Ch. 4 System Design & Implementation Ch. 6 Conclusion & Future Work	AI pipeline: data ingestion and feature engineering (yfinance fetch, the top-100 screener, the technical indicators), the sentiment engine (FinBERT, Finnhub consensus, price targets, news), and the FastAPI service. Backend: the modular-monolith foundation (EF contexts, Hybrid-Cache, outbox/inbox, MassTransit, Quartz), the Users module, and the Notifications module. Frontend: the authentication pages, the Market view, the Learning environment, Portfolios, Profile, and the app shell. Testing: the Users, Portfolio, and Notifications unit tests, the coverage tooling, and the defect log. Requirements / PM: authentication, notifications, and data requirements, the repository and template setup, and the backend and pipeline track.	The FastAPI pipeline service, the sentiment engine, the back-end platform, the Users and Notifications modules, the market and account UI, the unit-test suites and coverage report, and the Docker Compose deployment, plus the architecture, deployment, ingest-sequence, ERD, Gantt, and use-case UML diagrams.

Joint vs Individual Work

Most of the build was individual. Each module had a single owner who carried it from design through implementation and tests, so the code rarely had two hands on it. What we shared sat at the seams. The system architecture and the module boundaries were drawn together before either of us wrote a line, the integration points where the Recommendations module reaches into the pipeline and the Portfolio and Users public

APIs were paired on, and the Chapter 3 requirements and traceability matrix were agreed between us. After that the division held. Seif took the recommendation side: the model, the risk grading, the personalisation, and the screens and tests built on them. Yahia took everything that delivers and feeds it, from the infrastructure and authentication through the messaging, the market data, and the account pages. The steady point of overlap was reviewing each other's pull requests.

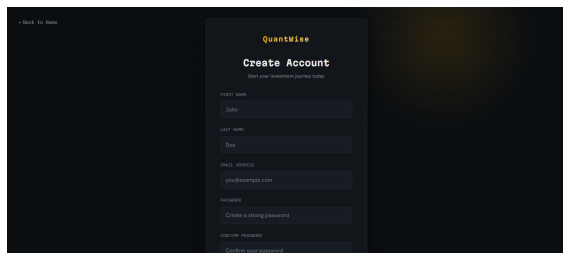
References

- [1] T. Fischer and C. Krauss, “Deep learning with long short-term memory networks for financial market predictions,” *European Journal of Operational Research*, vol. 270, no. 2, pp. 654–669, 2018.
- [2] O. B. Sezer, M. U. Gudelek, and A. M. Ozbayoglu, “Financial time series forecasting with deep learning: A systematic literature review: 2005–2019,” *Applied Soft Computing*, vol. 90, p. 106181, 2020.
- [3] J. Bollen, H. Mao, and X. Zeng, “Twitter mood predicts the stock market,” *Journal of Computational Science*, vol. 2, no. 1, pp. 1–8, 2011.
- [4] D. T. Araci, “FinBERT: Financial sentiment analysis with pre-trained language models,” 2019.
- [5] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proceedings of NAACL-HLT 2019*, 2019, pp. 4171–4186.
- [6] S. Wu, O. Irsoy, S. Lu, V. Dabrovolski, M. Dredze, S. Gehrmann, P. Kambadur, D. Rosenberg, and G. Mann, “BloombergGPT: A large language model for finance,” 2023.
- [7] A. Lopez-Lira and Y. Tang, “Can ChatGPT forecast stock price movements? Return predictability and large language models,” 2023.
- [8] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [9] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

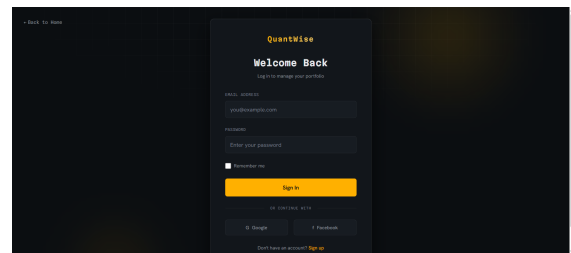
Appendices

[Appendix A ...; Appendix B ...]

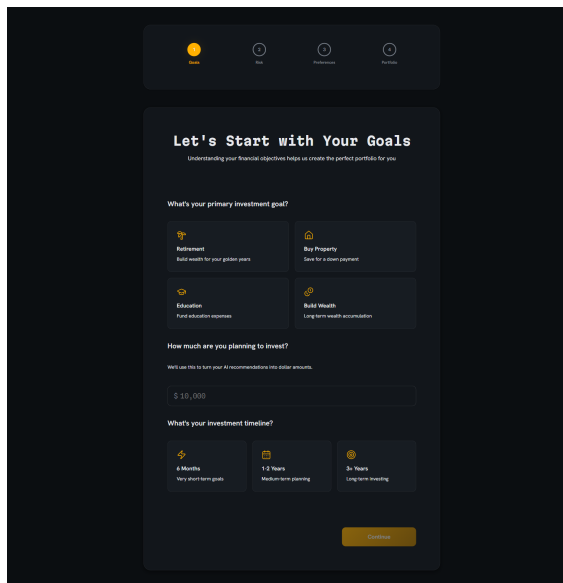
Appendix B — Interface Screenshots



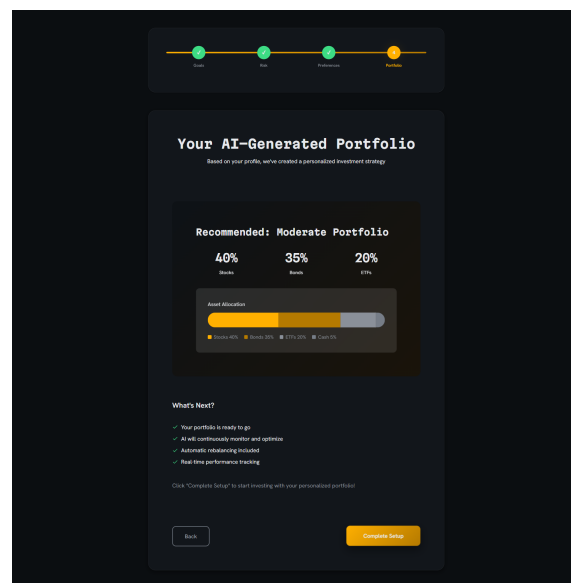
(a) Sign-up



(b) Login



(c) Onboarding — goals



(d) Onboarding — computed allocation

Figure 6.1: Authentication and onboarding.

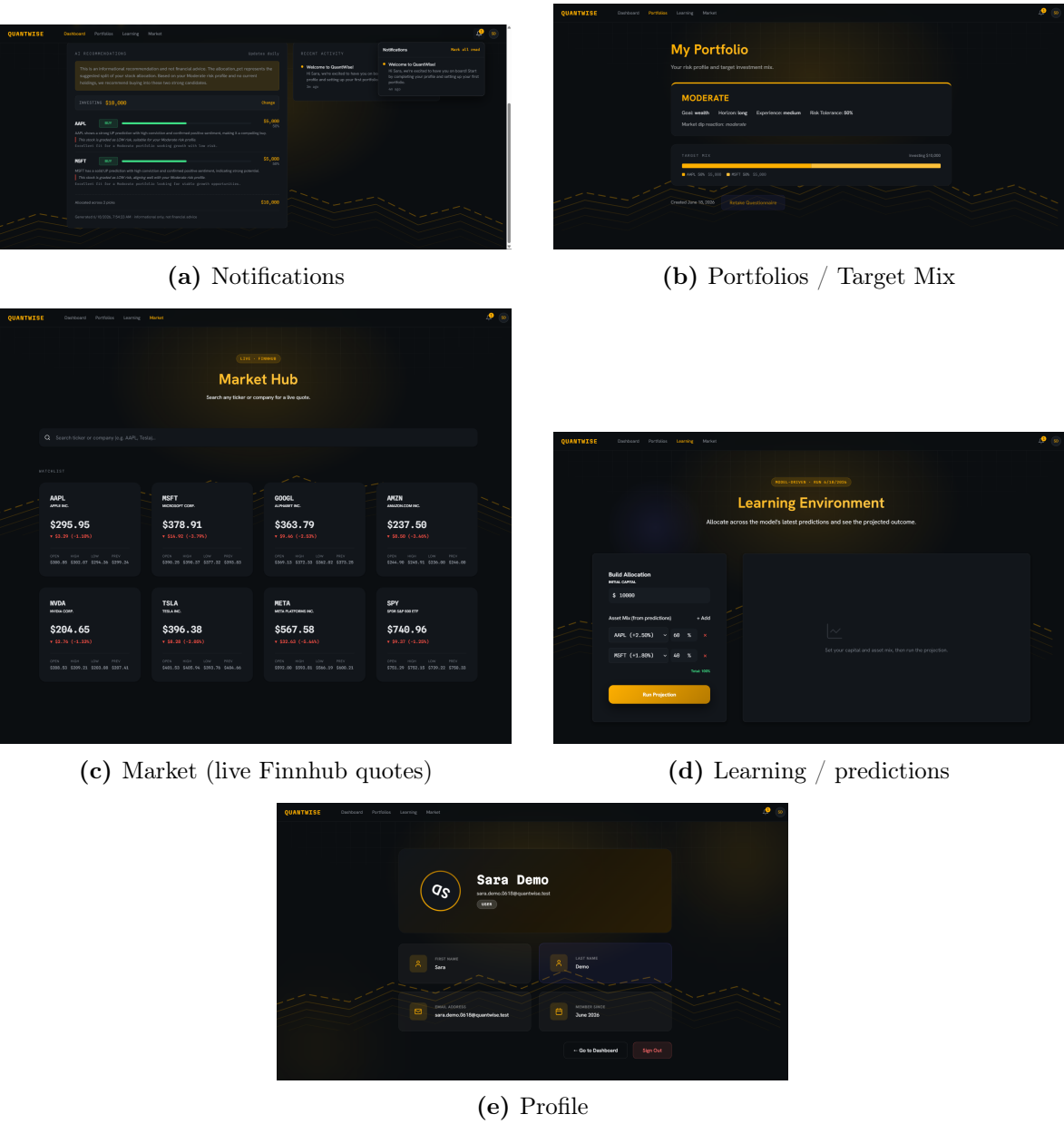


Figure 6.2: Dashboard, market, learning, and profile screens.